< Go to the original

SEGMENT ANYTHING 3D
POINT CLOUDS

Segment Anything 3D for Point Clouds: Complete Guide (SAM 3D)

How to build a semantic segmentation application for 3D point clouds leveraging SAM and Python

Florent Poux, Ph.D.
Follow
Towards Data Science a11y-light ~26 min read · December 13, 2023 (Updated: December 19, 2023) ·
Free: No

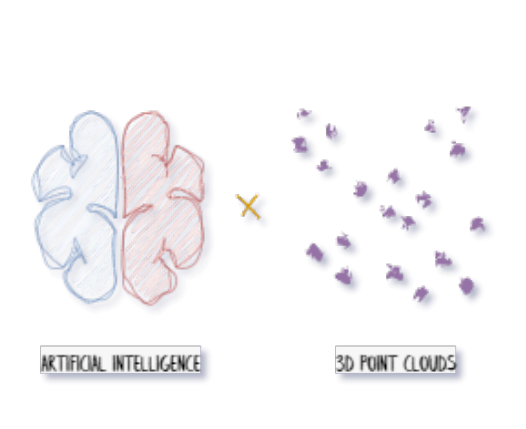
3D Python

How to build a semantic segmentation application for 3D point clouds leveraging SAM and Python. Bonus: code for projections and relationships between 3D points and 2D pixels.

Technological leaps are just plain crazy, especially looking at Artificial Intelligence (AI) applied to 3D challenges. Having the ability to leverage the latest cutting-edge research for advanced 3D applications is very empowering. Especially when looking at bringing human-level reasoning capabilities to a computer, there is a clear need to extract a formalized meaning from the 3D entities that we observe.

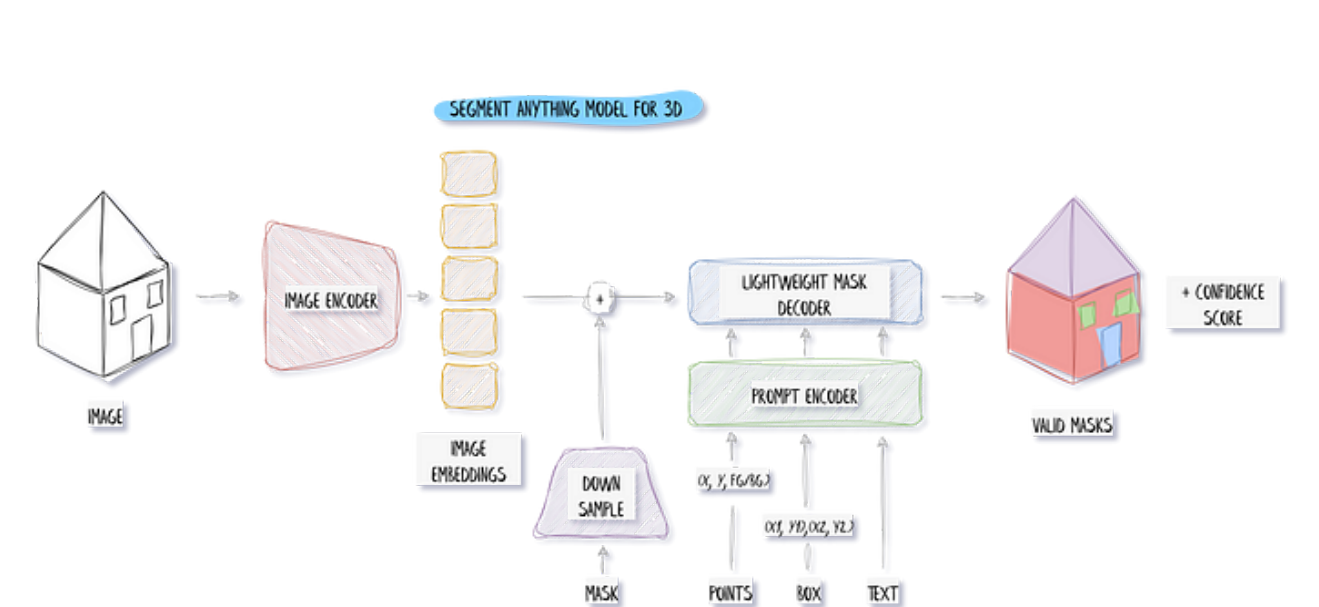
> In this tutorial, we are here to make sure that we can bind amazing AI advancements with 3D applications that make use of 3D Point Clouds. — 🐲 **Florent & Ville**

This is no easy feat, but once mastered, the fusion of 3D point clouds and deep learning gives birth to new dimensions of understanding and interpreting our visual world.



Artificial Intelligence ft. 3D point clouds. © F. Poux

Among these advancements, the Segment Anything Model is a recent beacon of innovation, especially for full automation without supervision.



The Segment Anything Model Architecture that we use for 3D Data. It comprises an image encoder, image embeddings, and some pre-processing operations to finally pass into the decoder and prompt encoder, giving the results as masks. © F. Poux

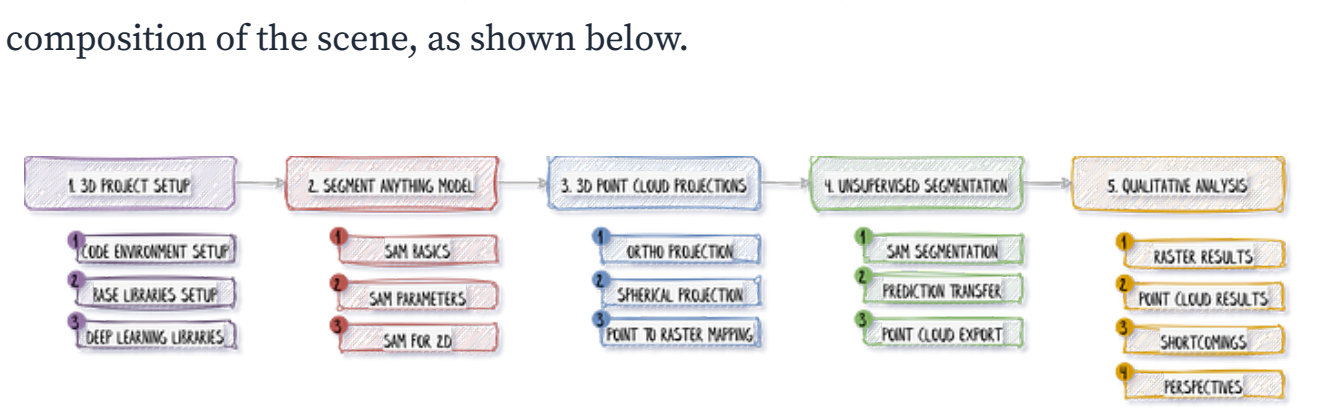
In this ultimate guide, we embark on a pragmatic voyage to explore this cutting-edge model, from its inception to its practical segmentation applications. But what is the objective here?

The Mission 🥷

Okay, it's time for the mission brief! You are now a multi-class member of your country's special forces, and you must find some dangerous materials hidden inside a specific building without ever being detected (here: the ITC building).

With your superb internet hacking skills, you manage to find the 3D scans for that part of the building you are interested in. You now need to find a way to define the path for your dangerous material recovery team quickly. After that, the team can proceed unnoticed to recover the materials, and you have saved the day!

After careful research and using your various skills, you develop a 3D data processing workflow that involves setting up a 3D Python code environment to process the 3D point cloud by using the Segment Anything Model to highlight the composition of the scene, as shown below.



The workflow for Segment Anything 3D. We have five main steps (3D Project Setup, Segment Anything Model, 3D Point Cloud Projections, Unsupervised Segmentation, and Qualitative Analysis) further refined in the substeps, as highlighted. © F. Poux

This will allow you to produce a 3D semantic map that will permit pinpointing the location of the materials within ninety minutes before the team is on-site. Are you ready?

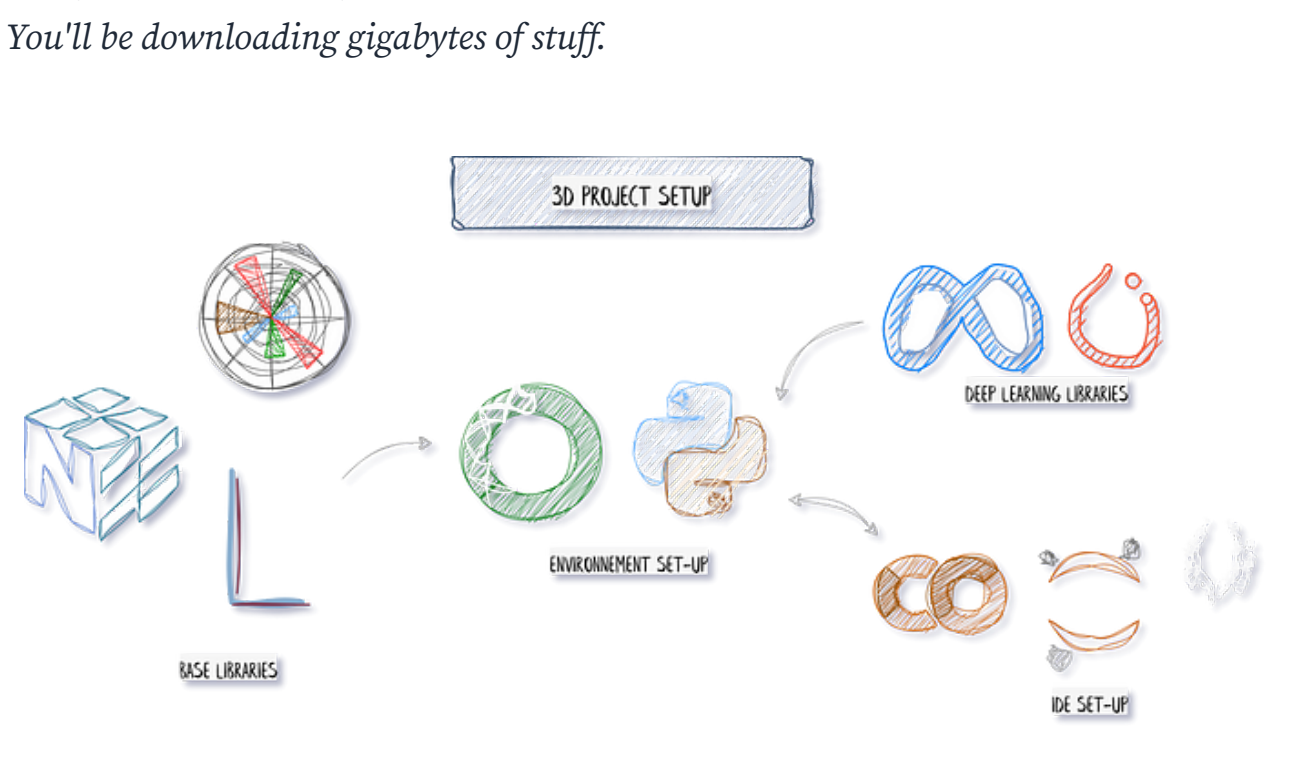
🎵 **Note to Readers**: *This hands-on guide is part of a* ***UTWENTE*** *joint work with co-authors* ***F. Poux*** *and* ***V. Lehtola****. We acknowledge the financial contribution from the digital twins @ITC -project granted by the ITC faculty of the University of Twente.*

1. 3D Project Setup

Before we dive into the marvels of the Segment Anything Model, it is crucial to establish a robust foundation. Setting up the appropriate environment ensures smooth sailing throughout our journey, allowing seamless experimentation and exploration.

At this stage, we want to ensure that our coding environment is correctly set with robust libraries. Ready?

🤠 **Ville:** *This is before the action starts. Please reserve an hour or two separately for this if you are doing it from scratch and, e.g., might need to update CUDA drivers. You'll be downloading gigabytes of stuff.*

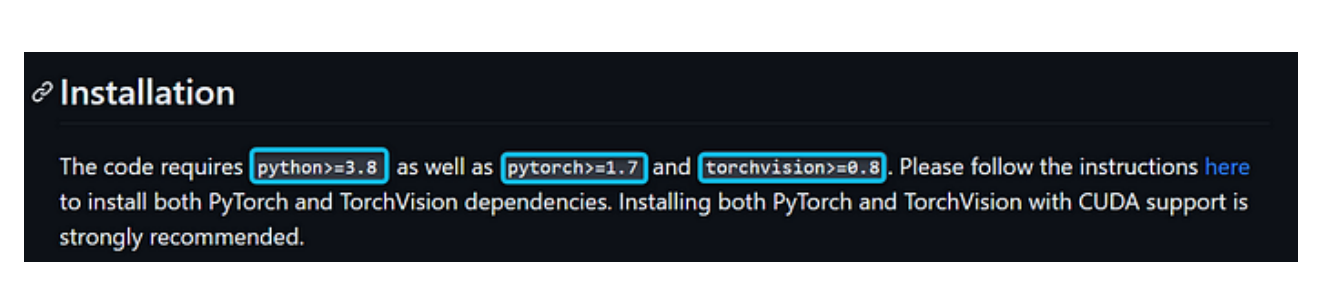


The 3D Project Setup. We first set up the environment, on which we attach base libraries, deep learning libraries, and the IDE Setup. © F. Poux

1.1. 3D Code Environment setup

It is time to get our hands in the dirt! We aim to use the Segment Anything Model to Semantically Segment a 3D Point Cloud. And that is no easy feat! So, of course, the first reflex is to check out the segment anything dependencies: Access SAM Github.

From there, we check out the necessary pre-requisites of the package:



Installation

The code requires `python>=3.8` as well as `pytorch>=1.7` and `torchvision>=0.8`. Please follow the instructions here to install both PyTorch and TorchVision dependencies. Installing both PyTorch and TorchVision with CUDA support is strongly recommended.

The dependencies highlighted in Segment Anything.

🦊 **Florent:** *Whenever you are dealing with deep learning libraries or deep learning new research code, it is essential to check out the dependencies and installation recommendations. Indeed, this will strongly influence the follow-up of your experiments and the time needed for replication.*

As you can see, we need to use the following library version:

```
python ≥ 3.8
pytorch ≥ 1.7
torchvision ≥ 0.8
```

Now that this is out, we will generate a virtual environment to ensure smooth sailing! If you want a detailed view of the process, I recommend you jump aboard the following guide:

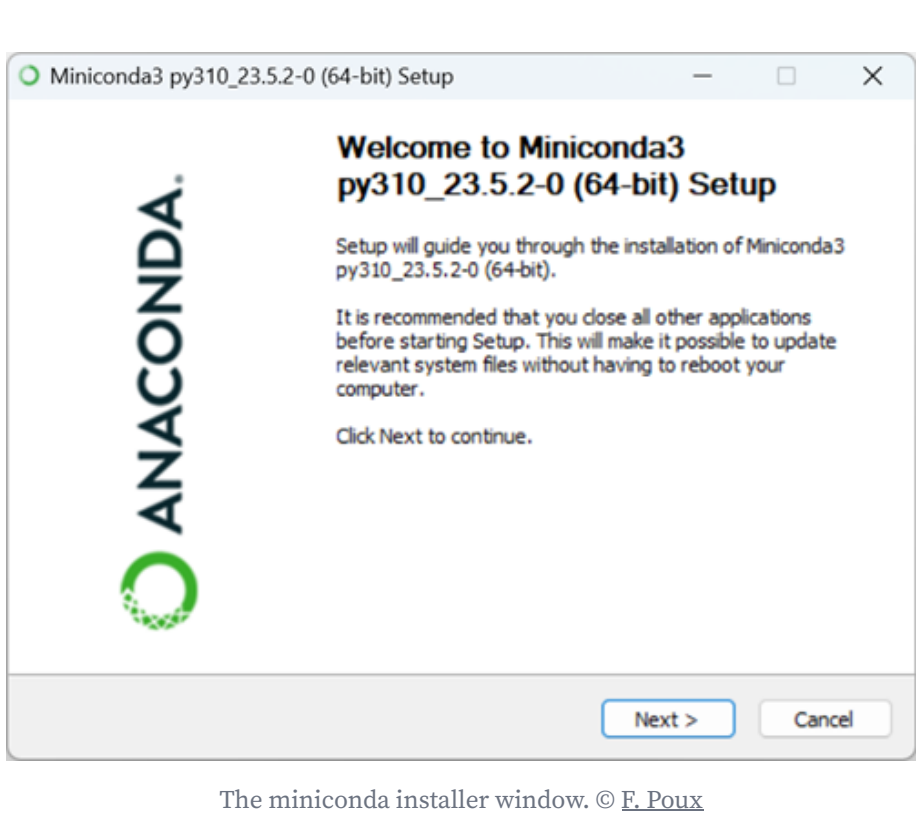
3D Python Workflows for LiDAR City Models: A Step-by-Step Guide
The Ultimate Guide to unlocking a streamlined workflow for 3D City Modelling Applications. The tutorial covers Python…
towardsdatascience.com

But, not to keep you high and dry, here is another strategy for a quick and lightweight setup using Miniconda.

💡 **Note:** *Miniconda is a free minimal installer for conda. It is a "miniature" version of Anaconda that includes only a minimal amount of dependencies. These are the conda package manager, a Python version, the packages they both depend on and other valuable packages like pip and zlib. This allows us only to install what we need in a lightweight manner.*

🤠 **Ville***: The cool stuff about virtual environments is that you can export it and run your code as-is on powerful Linux computing machines and superclusters! This is very handy for training networks!*

After downloading a version of Miniconda from here for your OS (I recommend you choose a Python 3.9 or 3.10 version to ensure proper compatibility with packages), you can install it following the various steps of the installation process.



Welcome to Miniconda3 py310_23.5.2-0 (64-bit) Setup

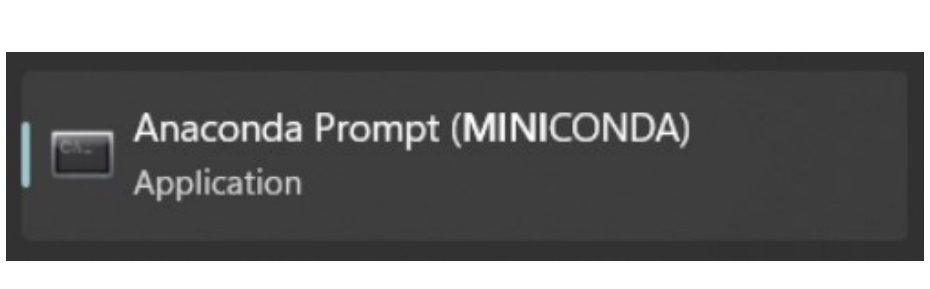
Setup will guide you through the installation of Miniconda3 py310_23.5.2-0 (64-bit).

It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.

Click Next to continue.

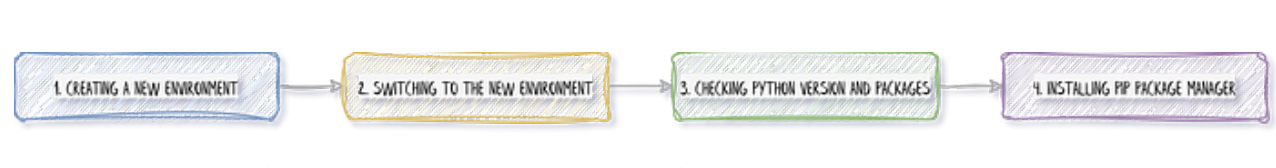
The miniconda installer window. © F. Poux

And that is it! You now have secured the most uncomplicated Python installation with the lightweight miniconda that will make isolating a controlled virtual environment super easy. Before moving on to the following steps, we launch miniconda with its command line access:



In Windows, just searching "miniconda" should yield this

Once in the Anaconda Prompt, we follow a simple four-step process to be up and running, as shown below.

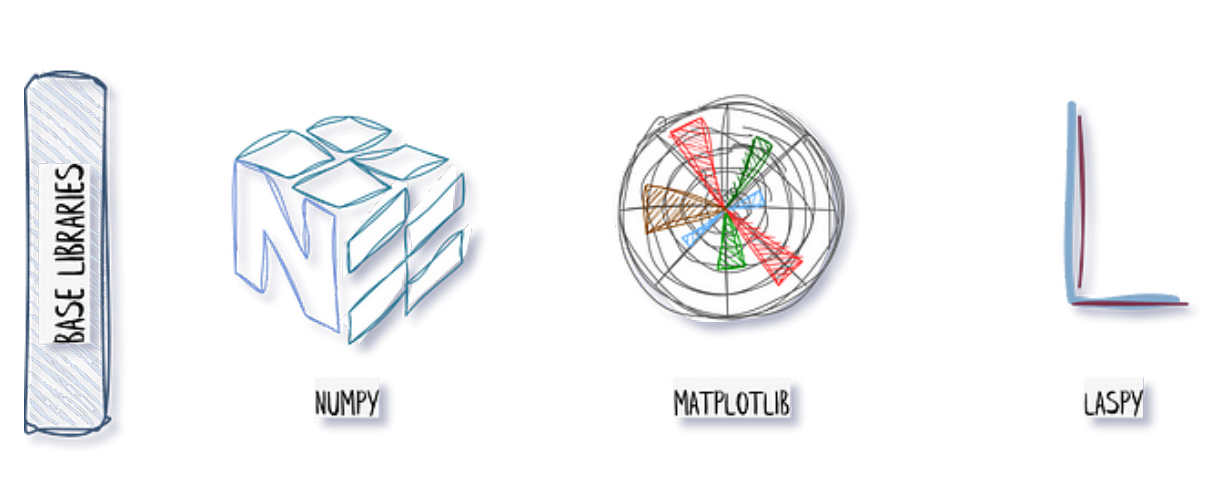

1. CREATING A NEW ENVIRONMENT
2. SWITCHING TO THE NEW ENVIRONMENT
3. CHECKING PYTHON VERSION AND PACKAGES
4. INSTALLING PIP PACKAGE MANAGER

The Workflow to set up a Python environment for 3D Segment Anything Model. © F. Poux

1. To create a new environment, we write the line: `conda create -n GEOSAM python=3.10`
2. To switch to the newly created environment, we write: `conda activate GEOSAM`
3. To check the Python version, `python --version`, and the installed packages: `conda list`. This should yield Python 3.10 and the list of base libraries, respectively.
4. To install pip in the new environment, we write: `conda install pip`

And that is it! We are now ready to move on installing the necessary libraries for playing with SAM.

1.2. Base library



The base libraries used in this tutorial (Numpy, Matplotlib, Laspy). © F. Poux

We now install our base libraries for using SAM: `NumPy`, `LasPy`, `OpenCV`, and `Matplotlib`. `NumPy` may be the most recommended library for numerical computations, `OpenCV` is used for computer vision tasks, `Laspy` deals with processing LiDAR data, and `Matplotlib` is a plotting and data visualization library in Python.

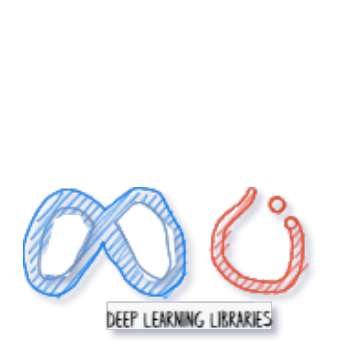
🦊 **Florent:** *These libraries are the base and robust cornerstones of any 3D project. If you want to deepen their understanding, I suggest you dive into this tutorial that explores its dark depths.*

To install these libraries, we can use pip in one single line:

```
pip install numpy matplotlib laspy opencv-python
```

This is great; it's time for the deep learning libraries setup!

1.2 Deep Learning Libraries



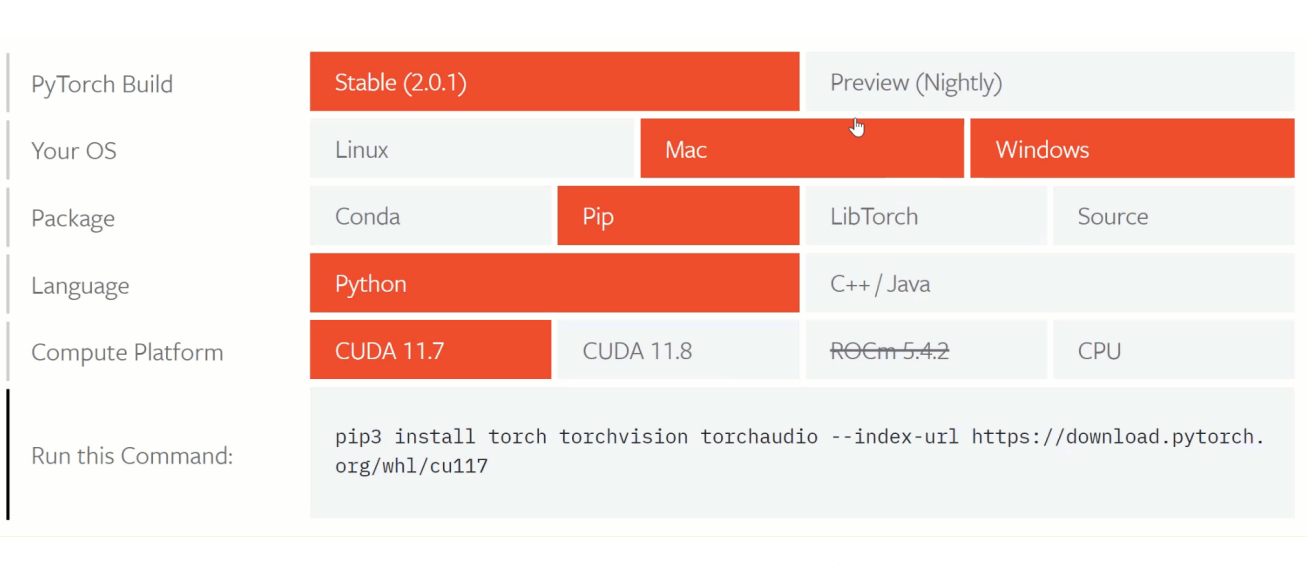
The deep learning libraries.

We will now look into installing deep-learning libraries. And, of course, the first one that we explore is my favorite one so far: Pytorch. Since its launch in 2017, Pytorch has improved its flexibility and hackability as a priority and performance as a close second. Therefore, today, using Pytorch for Deep Learning applications is excellent if you want (1) high-performance execution, (2) Pythonic internals, and (3) good abstractions for valuable tasks.

🦊 **Florent:** *Since 2017, Hardware accelerators (such as GPUs) have become ~15x faster in computing tasks. You can only guess what is to come in the next few years. Therefore, it is essential to be on the lookout for flexible libraries that can move quickly, even on refactoring "internals" to languages such as C++, like Pytorch does.*

🤠 **Ville***: SAM authors recommend using a GPU with 8GB memory. However, we give some tips on how to do the tutorial with less memory. Use them if you get 'MemoryError' or 'Out-of-bounds memory access' or 'Illegal memory access' messages. I got it working with 6GB.*

To install a relevant distribution of Pytorch without headaches figuring out how to install CUDA (which is not so straightforward), they made a straightforward web app that generates the code to copy and paste into your command line. For this, you can jump on this Pytorch Getting Started page and select the most relevant way to install your distribution, as shown below.



PyTorch Build	Stable (2.0.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.7	CUDA 11.8	~~ROCm 5.4.2~~	CPU

Run this Command:

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu117
```

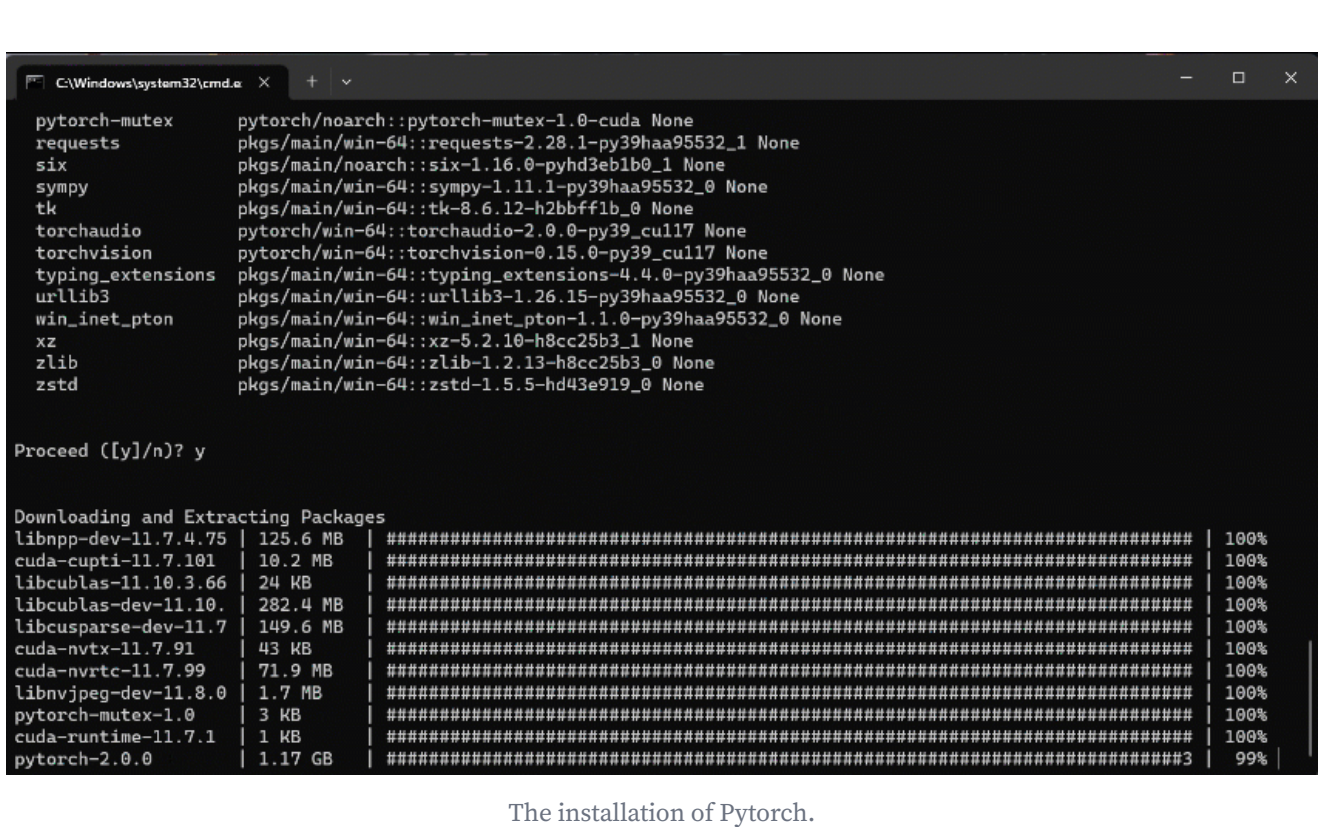
How to install Pytorch for your OS and configuration.

💡 **Note:** *We want to leverage our GPU. Therefore, it is essential to note that we want an installation with CUDA. But this is possible only if you have a Nvidia GPU at the time of writing. If not, you may want to use the CPU or switch to a Cloud computing service such as Google Colab.*

Therefore, our code line is the following:

```
conda install pytorch torchvision torchaudio pytorch-cuda=11.7 -c pytorch -c
```

This line will trigger the retrieval and installation of the necessary elements for Pytorch to function coherently.



```
  pytorch-mutex      pytorch/noarch::pytorch-mutex-1.0-cuda None
  requests           pkgs/main/win-64::requests-2.28.1-py39haa95532_1 None
  six                pkgs/main/noarch::six-1.16.0-pyhd3eb1b0_1 None
  sympy              pkgs/main/win-64::sympy-1.11.1-py39haa95532_0 None
  tk                 pkgs/main/win-64::tk-8.6.12-h2bbff1b_0 None
  torchaudio         pytorch/win-64::torchaudio-2.0.0-py39_cu117 None
  torchvision        pytorch/win-64::torchvision-0.15.0-py39_cu117 None
  typing_extensions  pkgs/main/win-64::typing_extensions-4.4.0-py39haa95532_0 None
  urllib3            pkgs/main/win-64::urllib3-1.26.15-py39haa95532_0 None
  win_inet_pton      pkgs/main/win-64::win_inet_pton-1.1.0-py39haa95532_0 None
  xz                 pkgs/main/win-64::xz-5.2.10-h8cc25b3_1 None
  zlib               pkgs/main/win-64::zlib-1.2.13-h8cc25b3_0 None
  zstd               pkgs/main/win-64::zstd-1.5.5-hd43e919_0 None


Proceed ([y]/n)? y


Downloading and Extracting Packages
libnpp-dev-11.7.4.75 | 125.6 MB  | ############################################################################ | 100%
cuda-cupti-11.7.101  | 10.2 MB   | ############################################################################ | 100%
libcublas-11.10.3.66 | 24 KB     | ############################################################################ | 100%
libcublas-dev-11.10. | 282.4 MB  | ############################################################################ | 100%
libcusparse-dev-11.7 | 149.6 MB  | ############################################################################ | 100%
cuda-nvtx-11.7.91    | 43 KB     | ############################################################################ | 100%
cuda-nvrtc-11.7.99   | 71.9 MB   | ############################################################################ | 100%
libnvjpeg-dev-11.8.0 | 1.7 MB    | ############################################################################ | 100%
pytorch-mutex-1.0    | 3 KB      | ############################################################################ | 100%
cuda-runtime-11.7.1  | 1 KB      | ############################################################################ | 100%
pytorch-2.0.0        | 1.17 GB   | ###########################################################################3 |  99%
```

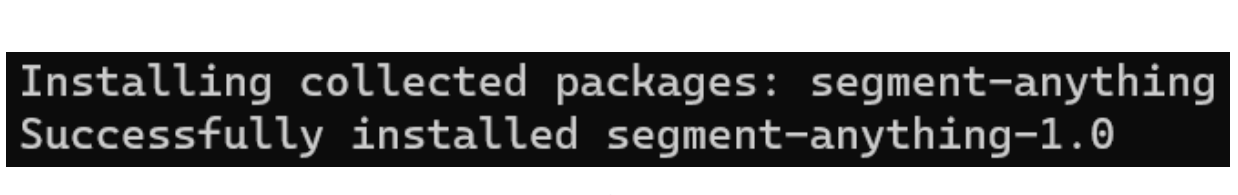
The installation of Pytorch.

The second deep-learning library that we want to use is Segment Anything. While Pytorch is being installed, we can download and install "software" that will make it easier for us to manage versions and access online libraries. This is **Git** and is accessible here: Git Website.

You can download and install git, and once the installation is finished, Pytorch should also be nicely installed in your environment. Therefore, to install segment-anything, we can write the line below:

```
pip install git+https://github.com/facebookresearch/segment-anything.git
```

This will again take some time until you get such a message below.

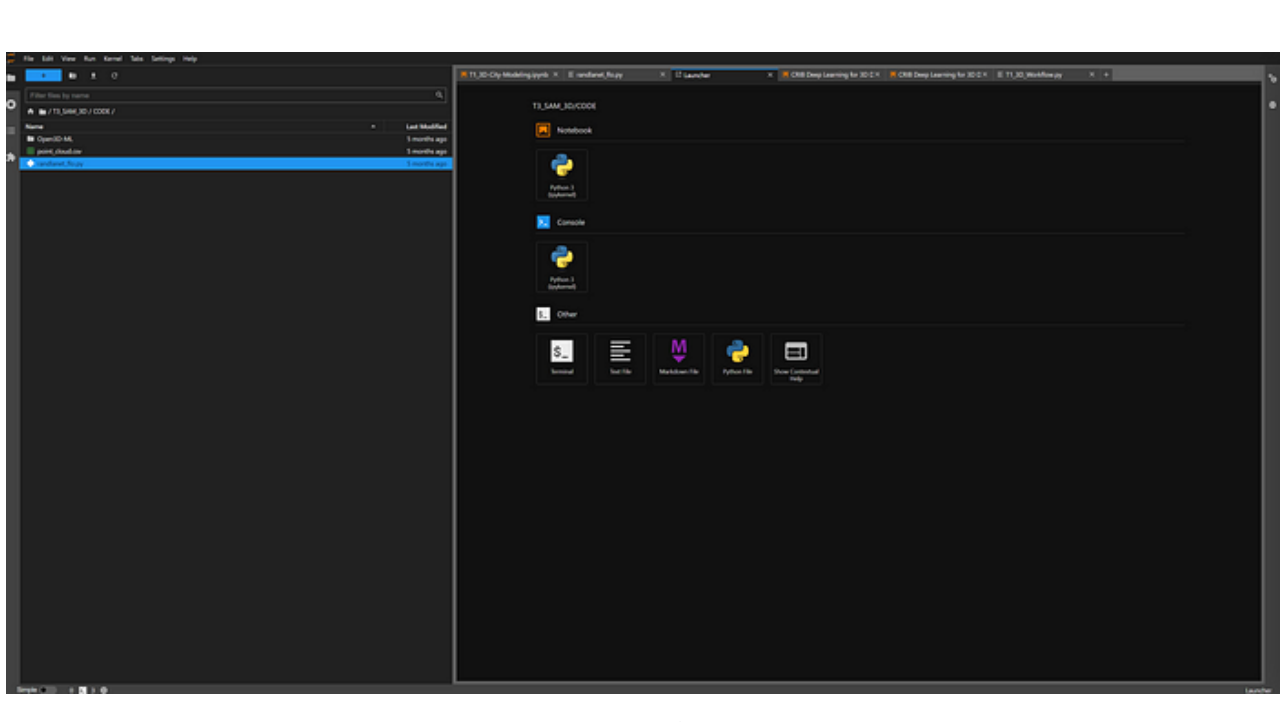


```
Installing collected packages: segment-anything
Successfully installed segment-anything-1.0
```

The CLI results of installing Pytorch.

At this stage, we have the base libraries as well as the deep learning libraries installed. Before using them, let us install an IDE to use everything smoothly.

4. Setting up an IDE



The Jupyter lab IDE after installation.

The last step of our setup is to install an IDE. We are still in the command line interface within the environment, and we type: `pip install jupyterlab`, which will install jupyterlab on our environment.

To use it in a defined local folder, we can first create a parent directory for our project (let us call it `SAM`), which will hold both a `CODE` folder and a `DATA` Folder. Once this is done, in the console, we change our pointer to the created directory by writing. `cd C://COURSES/POUX/SAM`.

We launch jupyterlab from this location by typing in the console: `jupyter lab`, which will open a new localhost page in your web browser (Chrome, Firefox, or Safari).

In Jupyter, you can create a notebook (.ipynb) and write in the first cell of the import statements to use all the installed packages:

```python
# The Base libraries
import numpy as np
import matplotlib.pyplot as plt
import cv2
import laspy

# The Deep Learning libraries
import torch
from segment_anything import sam_model_registry
from segment_anything import SamAutomaticMaskGenerator
```

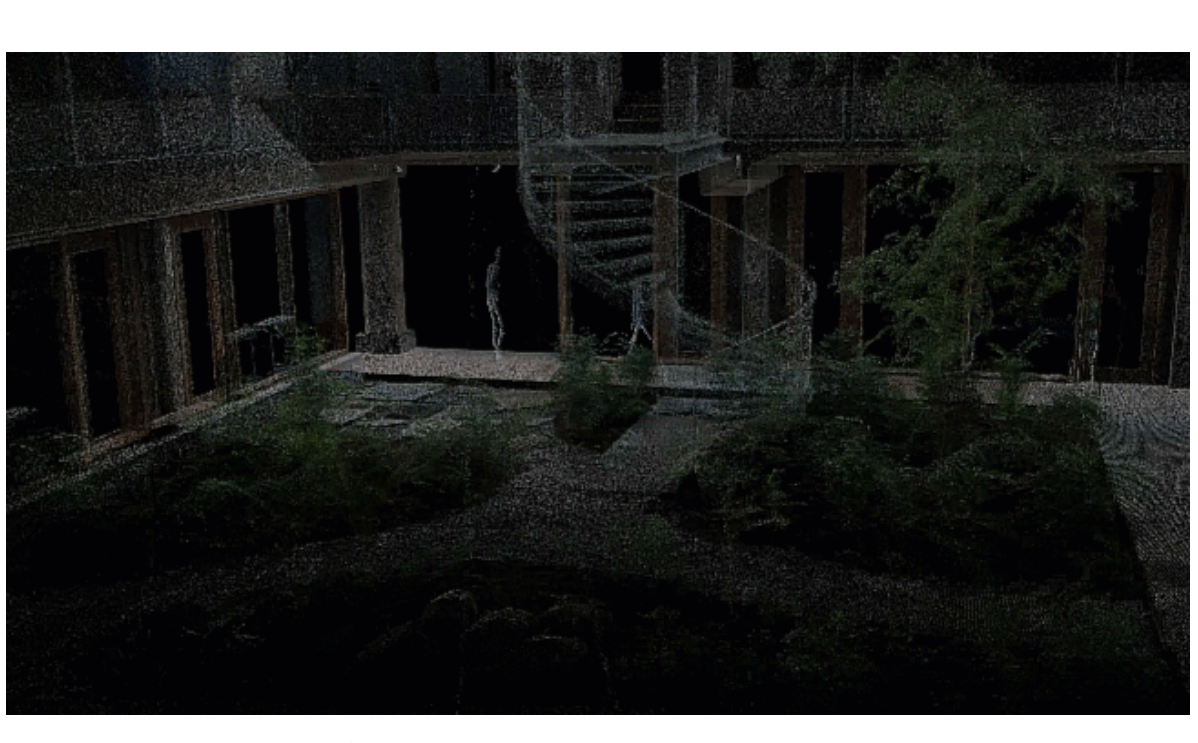
Alright! We are all set up. Before getting on the other steps of coding our model, now is just the time to retrieve our 3D Dataset.

5. 3D Dataset Curation

In previous tutorials, we illustrated point cloud processing and meshing over several 3D datasets, some of which use aerial LiDAR from the AHN4 LiDAR Campaign.

3D Deep Learning Python Tutorial: PointNet Data Preparation
The Ultimate Python Guide to structure large LiDAR point cloud for training a 3D Deep Learning Semantic Segmentation…
towardsdatascience.com

This time, we will use a dataset gathered using a Terrestrial Laser Scanner: the ITC of the University of Twente's new 2023 building, as shown below. It consists of an indoor green area with nice tricky foliage to assess after running the segmentation.



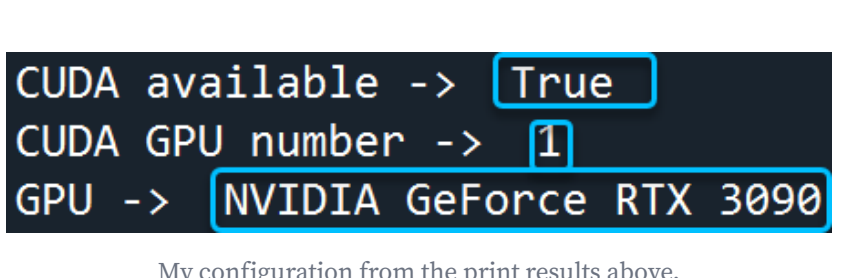
The 3D Point Cloud of the ITC UTwente new building, with its indoor "jungle". © F. Poux

You can download the data from the Drive Folder here: Guide Datasets (Google Drive), and put it in your folder that holds datasets (in my case, "DATA").

At this process stage, we have a nice warm coding setup, with all the necessary libraries in a lightweight, isolated GEOSAM conda environment.

🦊 **Florent**: Great job so far! If you are eager to run some tests to check that Pytorch is working as it should, i.e., CUDA is recognized, you can write these lines of code:

```python
import torch
print('CUDA available -> ', torch.cuda.is_available())
print('CUDA GPU number -> ', torch.cuda.device_count())
print('GPU -> ', torch.cuda.get_device_name())
```



```
CUDA available -> True
CUDA GPU number -> 1
GPU -> NVIDIA GeForce RTX 3090
```

My configuration from the print results above.

It is now time to segment stuff!

2. Setting up the Segment Anything Model

At the heart of our little adventure lies the Segment Anything Model, a powerful creation with excellent potential for 3D point cloud semantic segmentation. With its innovative architecture and training process, this model is the perfect candidate to be tested on indoor applications. Let us first play around with its core concepts.

2.1. Segment Anything Basics

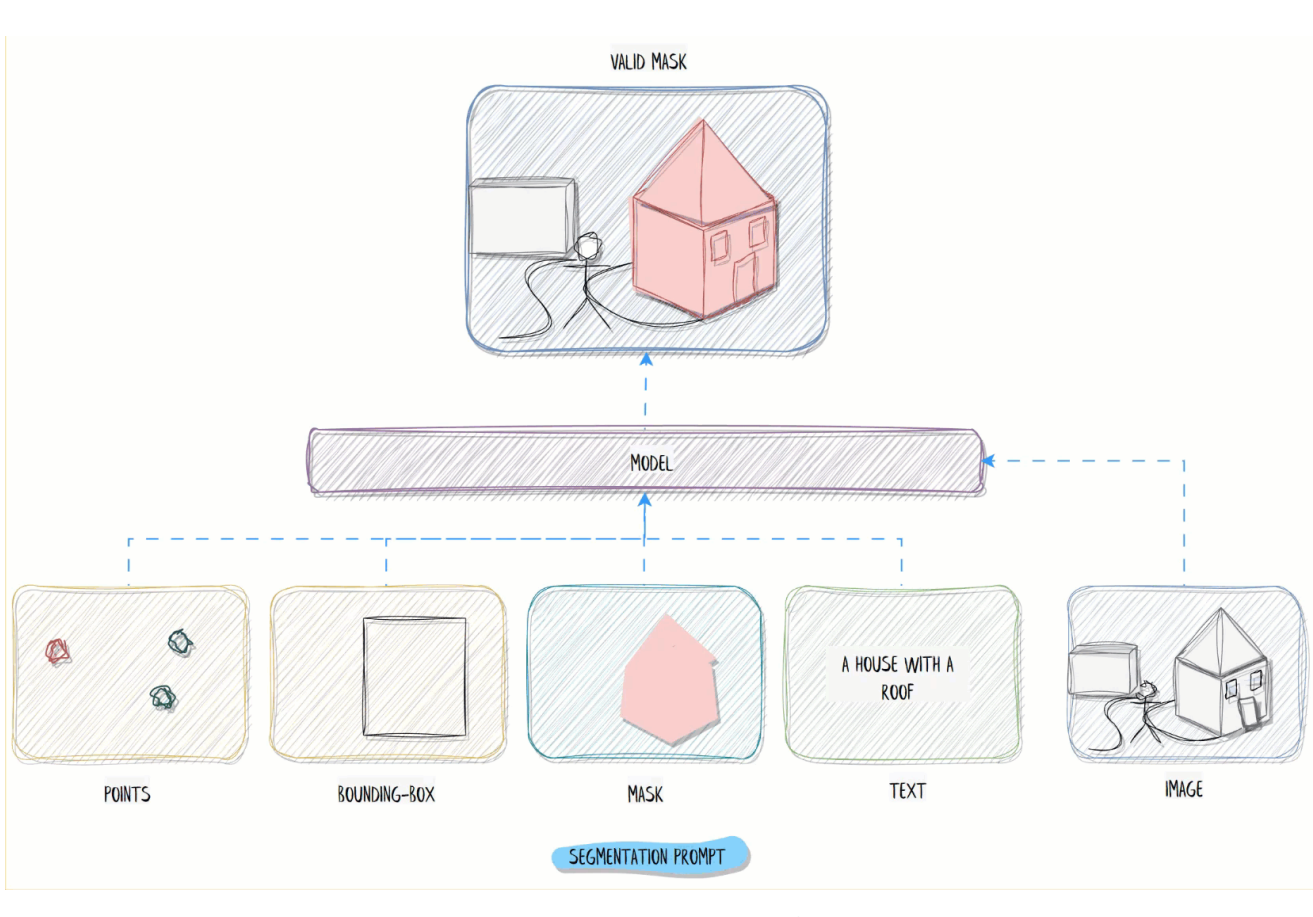
MetaAI has delved into the fascinating realm of Natural Language Processing (NLP) and computer vision with their Segment Anything Model, which enables **zero-shot** and **few-shot learning** on novel datasets and tasks using foundation models.

🦊 **Florent**: *Okay, there are a lot of swear words, I admit. For clarity concerns, here is my tentative to summarize each complex terminology. Zero-shot learning refers to the ability to recognize something without having seen it (seen it zero times). Somewhat similarly, few-shot learning uses a limited number of labeled examples for each new class, and the goal is to make predictions for new classes based on just these few examples of labeled data.*

🤠 **Ville**: *Also, a so-called **foundation model** is a model that is trained on a lot and a lot of data at a scale. it is so large that it can be adapted to various tasks from different scenarios.*

Let's break this down for you:

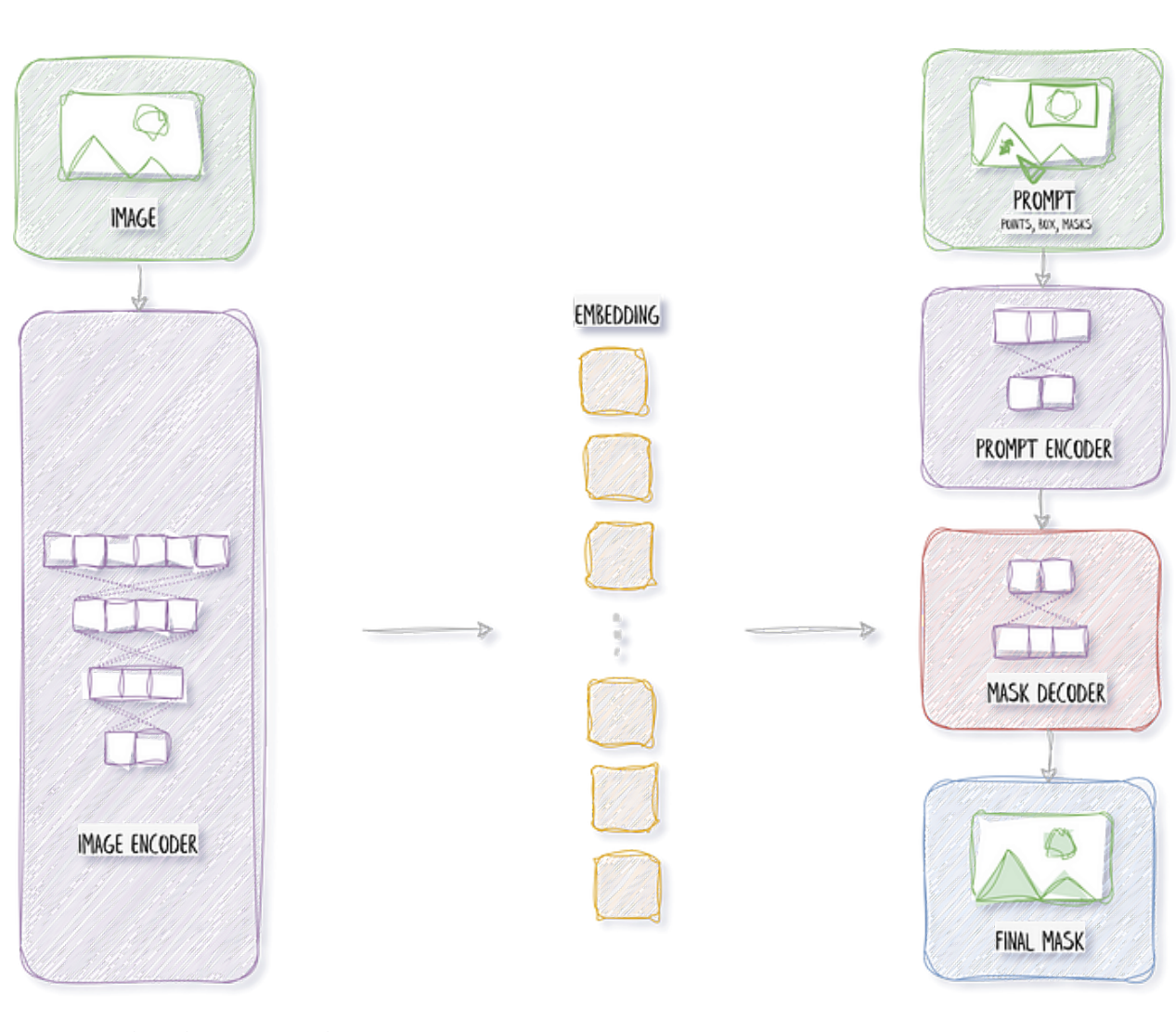
Overall, the SAM "AI" algorithm can significantly reduce the human effort required for image segmentation. To do so, you provide the model with foreground/background points, a rough box or mask, some text, or any other input that indicates what you want to segment in an image. The Meta AI team has trained the Segment Anything Model to generate a proper segmentation mask. This mask is the model's output and should be a suitable mask to delineate one of the things that the prompt might refer to. For instance, if you indicate a point on the roof of the house, the output should correctly identify whether you meant the roof or the house.



How does the Segment Anything Model (SAM) work? Explanation of the segmentation prompt to generate valid masks (case of a house). © F. Poux

This segmentation task can then serve for model pre-training and guiding solutions for various downstream segmentation problems.

On the technical side, what we call an image encoder creates a unique embedding (representation) for each image, and a lightweight encoder swiftly transforms any query into an embedding vector. These two data sources are merged using a (lightweight) mask decoder to predict segmentation masks, as shown below.



Flowchart of the functioning of the Segment Anything Model. The image goes through the image encoder. Then it is embedded, to finally be combined after using a prompt followed by a prompt encoder, to generate final masks for our 3D point clouds. © F. Poux

This effective architecture, combined with a massive scale training phase, allows the Segment Anything Model to reach four milestones:

- **Effortless Object Segmentation** 🔥: With SAM, users can effortlessly segment objects by simply selecting the points they want to include or exclude from the segmentation. You can also use a bounding box as a cue for the model.
- **Handling Uncertainty** 🔥: SAM is equipped to handle situations with uncertainty about the object to be segmented. It can generate multiple valid masks, which is crucial for solving real-world segmentation challenges effectively.
- **Automatic Object Detection and Masking** 🔥: SAM makes automatic object detection and masking a breeze. It simplifies these tasks, saving you time and effort.
- **Real-time Interaction** 🔥: Thanks to precomputed image embeddings, SAM can instantly provide a segmentation mask for any prompt. This means you can have real-time interactions with the model.

Now that this is out of the way, are you ready to use it?

2.1. SAM Parameters

The SAM model can be loaded with three different encoders: ViT-B, ViT-L, and ViT-H. ViT-H gives better results than ViT-B but has only marginal gains over ViT-L.

```
|     Encoder          |   #parameters    |     Speed      |   Quality    |
|----------------------|------------------|----------------|--------------|
|   ViT-B   (basic)    |       91M        |    Fastest     |   Low        |
|   ViT-L   (large)    |       308M       |    Fast        |   High       |
|   ViT-H   (huge)     |       636M       |    Slow        |   Highest    |
```

🤠 **Ville**: *To help with the choice, I tested ViT-B on NVIDIA GeForce GTX 1650, 6 Gb VRAM with Win11.*

These three encoders have different parameter counts that give a bit more freedom to tune an application. ViT-B (the smallest) has 91 Million parameters, ViT-L has 308 Million parameters, and ViT-H (the biggest) has 636 Million parameters.

This difference in size also influences the speed of inference, so this should help you decide the encoder for your specific use case. Following this guide, we will get with the heavy artillery: The ViT-H, with a Model Checkpoint that you can download from Github (2.4 Gb) and place in your current parent folder, for example.

This is where we can define two variables to make your code a bit more flexible afterward:

```
MODEL = "../../MODELS/sam_vit_h_4b8939.pth"

#You can run the line below to test if you have the ability to leverage CUDA
torch.cuda.is_available()

#Choose between cpu or cuda training. For cpu, input 'cpu' instead 'cuda:0'
USED_D = torch.device('cuda:0')
```

From there, we can initialize our SAM model with the following two lines of code:

```
sam = sam_model_registry["vit_h"](checkpoint = MODEL)

#Cast your model to a specific device (cuda or cpu)
sam.to(device = USED_D)
```

And we are all set up! Maybe one last step, trying to see how it performs on a random image that you have on your desktop?

2.2 Performances on 2D images

Let us test if all works as expected on a random image. We are interested in geospatial applications, so I go to Google Earth and zoom in on a spot of interest:

Selection of an imagery dataset from Biscarosse. © F. Poux

🦊 **Florent**: *This spot is biased, right? Hopefully, this gives you a bit of French holiday vibes, which you are proud to take followed by a marvelous year full of exciting projects*!

From there, I take a screenshot of a zone of interest:

The image dataset of a zone of Biscarosse plage. © F. Poux

and I load the image into memory with openCV:

```
#When loading an image with openCV, it is in bgr by default
loaded_img = cv2.imread("../DATA/biscarosse.jpg")

#Now we get the R,G,B image
image_rgb = cv2.cvtColor(loaded_img, cv2.COLOR_BGR2RGB)
```

🌳 **Note**: *As you can see, by default, OpenCV loads an image by switching to Blue, Green, and Red channels (BGR) that we order as RGB with the second line and store in the* `image_rgb` *variable.*

Now, it is time for us to apply SAM on the image with two lines of code:

```
mask_generator = SamAutomaticMaskGenerator(sam)
result = mask_generator.generate(image_rgb)
```

In around 6 seconds, this returns us a list filled with dictionaries, each representing a mask for a specific object automatically extracted, accompanied by its scores and metadata. For a detailed view, the result is a list of dictionaries where each `dict` holds the following information:

- `segmentation` : this brings out the mask with `(W, H)` shape (and `bool` type), where `W` (width) and `H` (height) target the original image dimensions;
- `area` : this is the area of the mask expressed in pixels
- `bbox` : this is the boundary box detection in `xywh` format
- `predicted_iou` : the model's prediction IoU metric for the quality of the mask.
- `point_coords` : This is a list of the sampled input points that were used to generate the mask
- `stability_score` : The stability score is an additional measure of the mask quality. Check out the paper for more details 😉
- `crop_box` : this is a list of the crop_boxe coordinates used to generate this mask in `xywh` format (it may differ from the Bounding-Box)

Now that you have a better idea about what we are dealing with, to check out the results, we can plot the masks on top of the image with the following function:

```
def sam_masks(anns):
    if len(anns) == 0:
        return
    sorted_anns = sorted(anns, key=(lambda x: x['area']), reverse=True)
    ax = plt.gca()
    ax.set_autoscale_on(False)
    c_mask=[]
    for ann in sorted_anns:
        m = ann['segmentation']
        img = np.ones((m.shape[0], m.shape[1], 3))
        color_mask = np.random.random((1, 3)).tolist()[0]
        for i in range(3):
            img[:,:,i] = color_mask[i]
        ax.imshow(np.dstack((img, m*0.8)))
        c_mask.append(img)
    return c_mask
```

🦊 **Florent**: *I admit, this is a bit blunt. But what happens in this function is that I will sort out the masks by their area to plot them with a random color on top of the image with a transparency parameter.*

🤠 **Ville**: *Memory errors can ruin French holiday vibes! Remember the Google Colab option too! If rebooting does not solve the issue and allocated memory is too high, the following piece of code clears the GPU memory of extra allocations. Use it to address memory problems.*

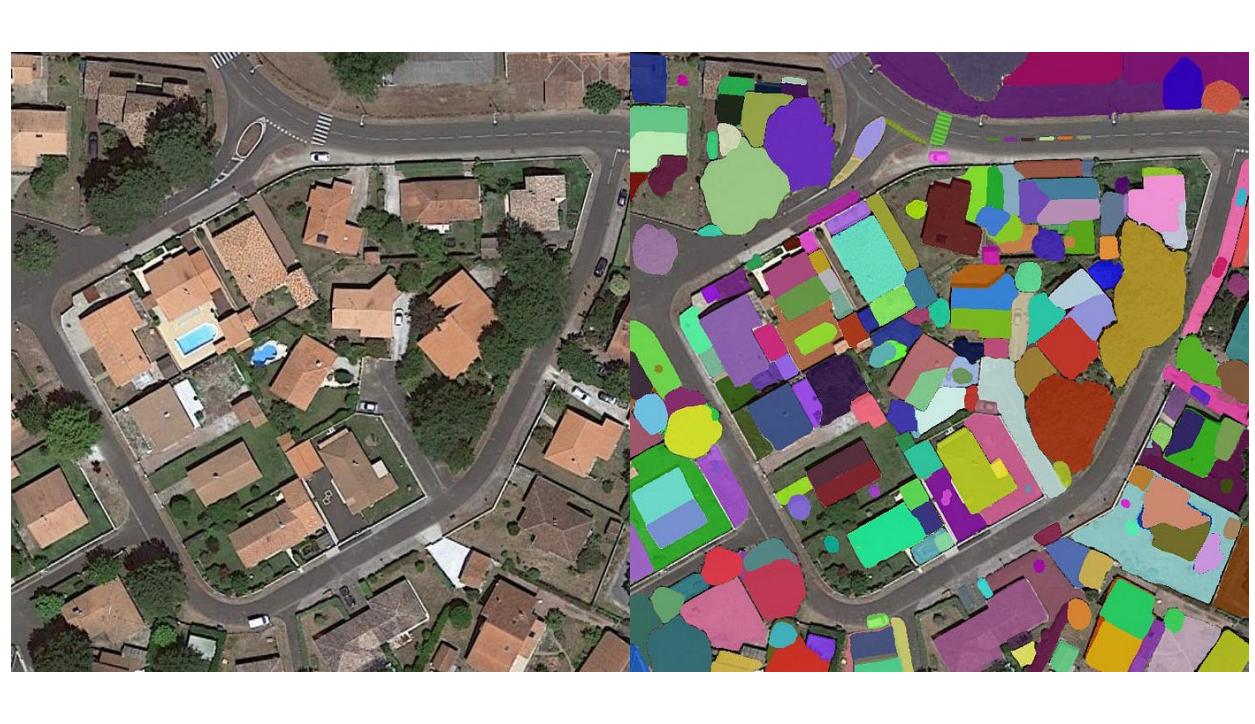
```
print('Mem allocated by other programs: ', torch.cuda.memory_allocated(), 'r
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:512"
import gc
gc.collect()
torch.cuda.empty_cache()

If the GPU memory is not freed enough, try rebooting your (Windows) computer
ALSO, try using the following line if memory problems persist
mask_generator = SamAutomaticMaskGenerator(sam, points_per_batch=16)
```

If the GPU memory is not freed enough, try rebooting your (Windows) computer. Also, try using the following line if memory problems persist: `mask_generator = SamAutomaticMaskGenerator(sam, points_per_batch=16)`

Now, to plot and export the image, we write the following:

```
fig = plt.figure(figsize=(np.shape(image_rgb)[1]/72, np.shape(image_rgb)[0]/
fig.add_axes([0,0,1,1])
plt.imshow(image_rgb)
color_mask = sam_masks(result)
plt.axis('off')
plt.savefig("../test_result.jpg")
```

Which results in:



So, already at this stage, we have exciting results, and SAM is working really nicely! For example, you can see that almost all roofs are part of segments and that the three pools (2 blue and one green) are also part of segments. Therefore, this could well be a starting point for complete automatic detection

🦊 **Florent**: *You may run into Memory Errors depending on your computer setup while plotting the masks. In this case, loading a lighter SAM model should solve your problem.* 😉

Now that we have a working SAM setup, let us apply all this hard-earned know-how to 3D point clouds.

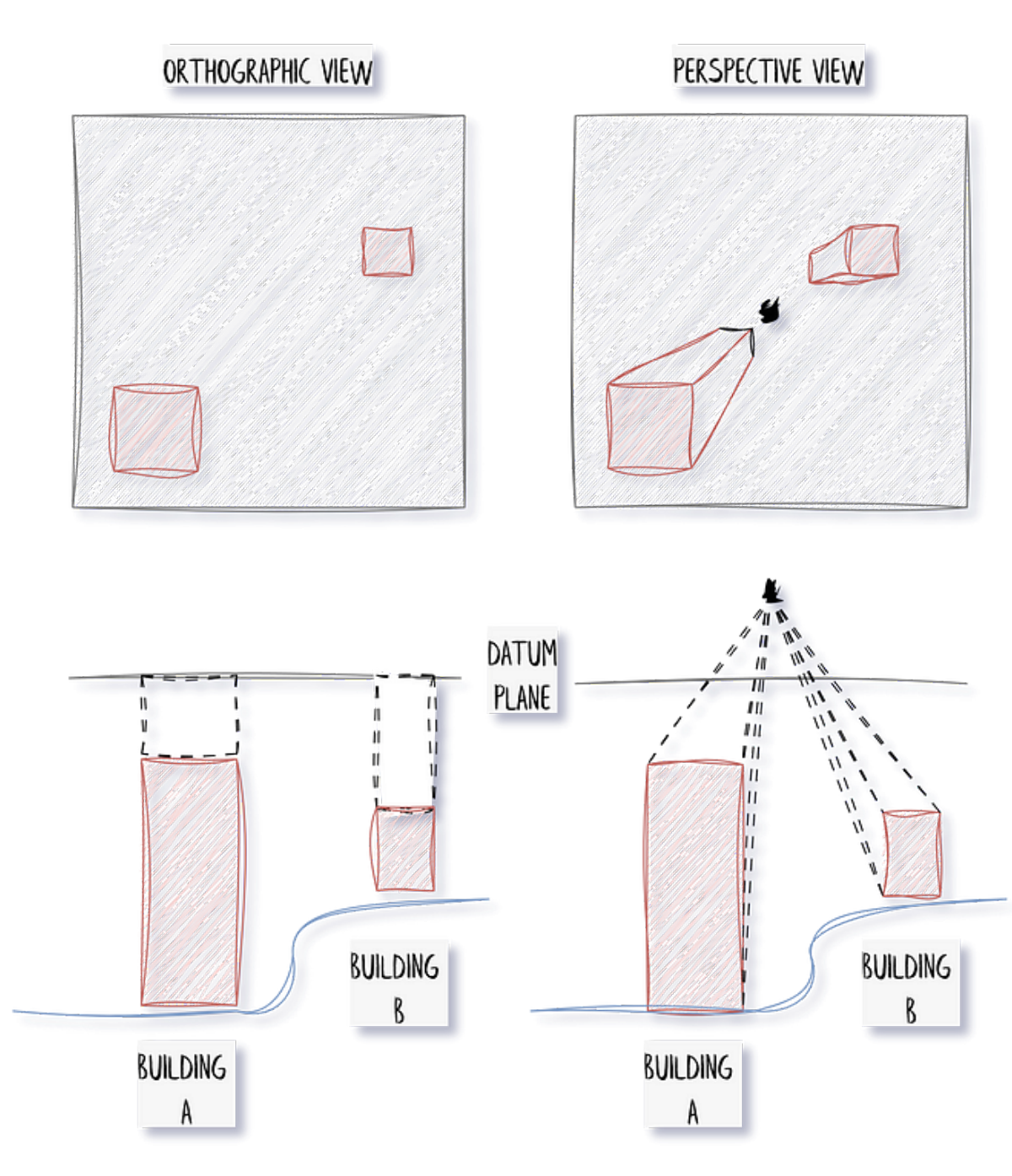
3. 3D Point Cloud to Image Projections

To make sense of the complex 3D world, we delve into the art of point cloud projection. Through techniques like ortho and spherical projections, we bridge the gap between dimensions, enabling us to visualize the intricacies of the point cloud in a 2D realm, which is the input needed for SAM. Point cloud mapping adds a layer of understanding to this projection process.

3.1 Ortho Projection: Flattening Dimensions, Expanding Insights

Let us look at the transformative technique of Ortho Projection. This method serves as an excellent bridge between the multi-dimensional complexities of 3D point clouds and the comprehensible world of 2D images. Through Ortho Projection, we "flatten" dimensions but also unveil a direct way to manage segmentation with SAM.

The idea is basically to generate a top-down view plane and generate an image that is not constrained by a single perspective. You could see ortho-projection as a process of pushing visible points from the point cloud (highest ones) onto the plane that holds the empty image to fill all the necessary pixels just above those points. You can see the difference from a perspective view, as illustrated below.



Explanation of the difference between Orthographic View, and Perspective View for 3D Projections. The Perspective View is linked to a single point of view that skews dimensions. © F. Poux

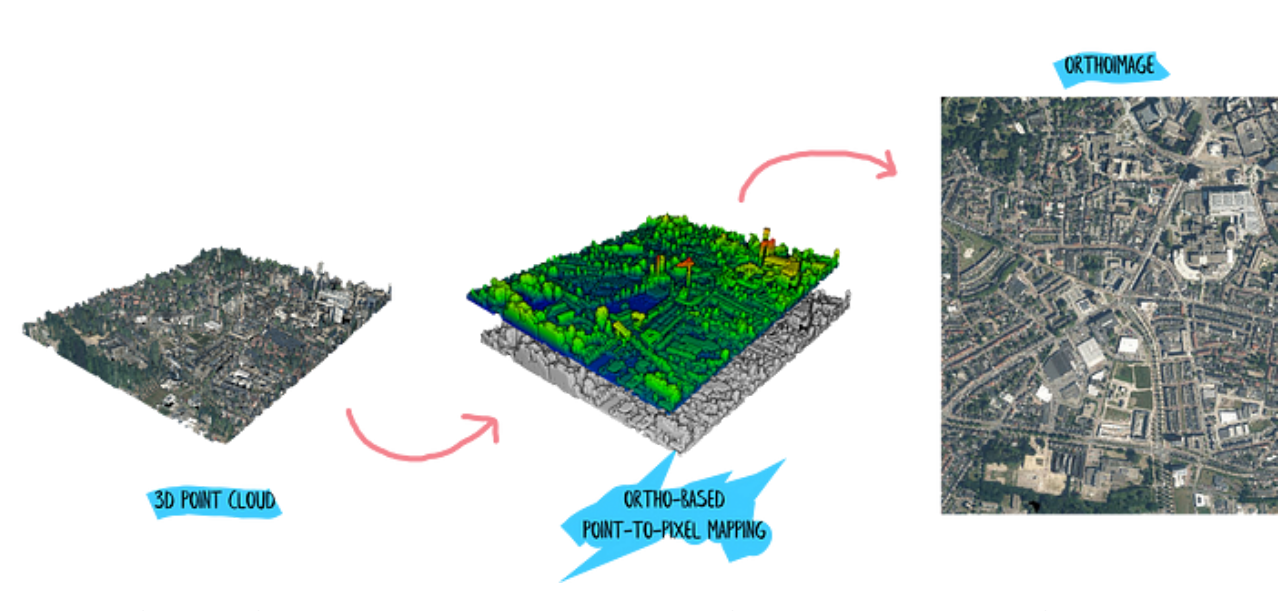
To work out this process, we can define a 3D-to-2D projection function that would take the points of a point cloud alongside its color and a wanted resolution to compute the ortho-projection and return an orthoimage from the point cloud. This would translate into the following:

```
def cloud_to_image(pcd_np, resolution):
    minx = np.min(pcd_np[:, 0])
    maxx = np.max(pcd_np[:, 0])
    miny = np.min(pcd_np[:, 1])
    maxy = np.max(pcd_np[:, 1])
    width = int((maxx - minx) / resolution) + 1
    height = int((maxy - miny) / resolution) + 1
    image = np.zeros((height, width, 3), dtype=np.uint8)
    for point in pcd_np:
        x, y, *_ = point
        r, g, b = point[-3:]
        pixel_x = int((x - minx) / resolution)
        pixel_y = int((maxy - y) / resolution)
        image[pixel_y, pixel_x] = [r, g, b]
    return image
```

Great, now it is time for a test, do you agree? To do so, let us load a point cloud dataset, transform it to a numpy array, apply the function, and export an image of this point cloud:

```
#Reading the point cloud with laspy
pcd = laspy.read("../DATA/34FN2_18.las")

#Transforming the point cloud to Numpy
pcd_np = np.vstack((pcd.x, pcd.y, pcd.z, (pcd.red/65535*255).astype(int), (p

#Ortho-Projection
orthoimage = cloud_to_image(pcd_np, 1.5)

#Plotting and exporting
fig = plt.figure(figsize=(np.shape(orthoimage)[1]/72, np.shape(orthoimage)[0
fig.add_axes([0,0,1,1])
plt.imshow(orthoimage)
plt.axis('off')
plt.savefig("../DATA/34FN2_18_orthoimage.jpg")
```

This permits us to obtain the following:



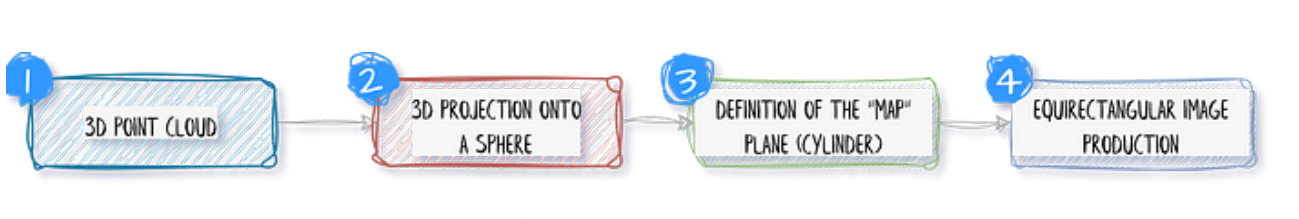
The workflow to go from 3D Point Clouds to Orthoimages. We first project the point cloud following an ortho-projection mode, then we make sure to include a point-to-pixel mapping for back-projection. © F. Poux

Let us move on to spherical projections

3.2 3D Point Cloud Spherical Projection

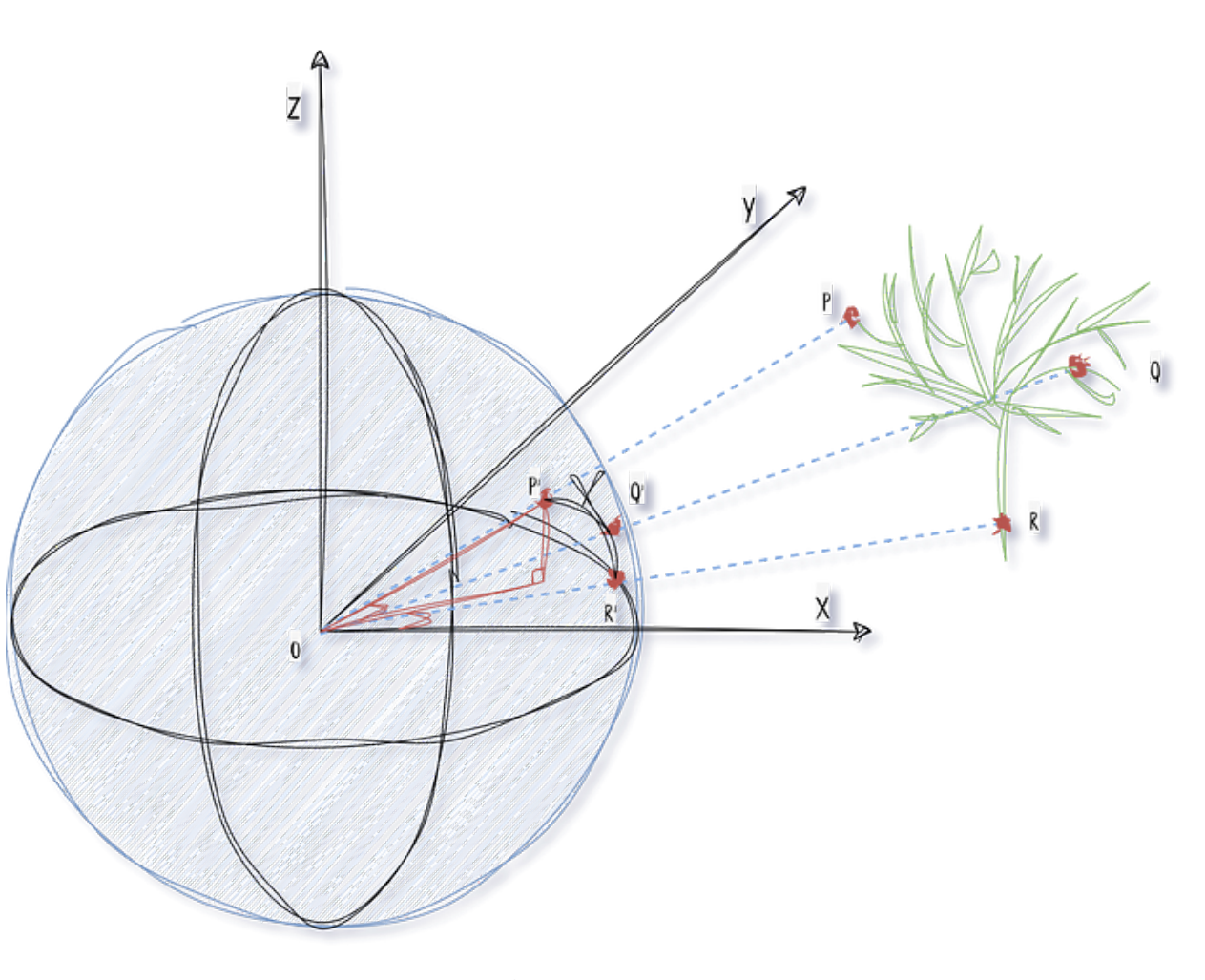
Our journey takes an intriguing turn as we encounter Spherical Projection. This technique offers a unique perspective, enabling us to visualize the data by "simulating" a virtual scan station. To do just this, we proceed in four steps by: (1) Considering the 3D Point Cloud, (2) Projecting these points onto a sphere, (3) defining a geometry that will retrieve the pixels, (4) "flattening" this geometry to produce an image.

🤠 **Ville**: *Spherical projection is like being inside a 3D point cloud and taking a 360-degree photo of what you see.*


1
3D POINT CLOUD
2
3D PROJECTION ONTO A SPHERE
3
DEFINITION OF THE "MAP" PLANE (CYLINDER)
4
EQUIRECTANGULAR IMAGE PRODUCTION

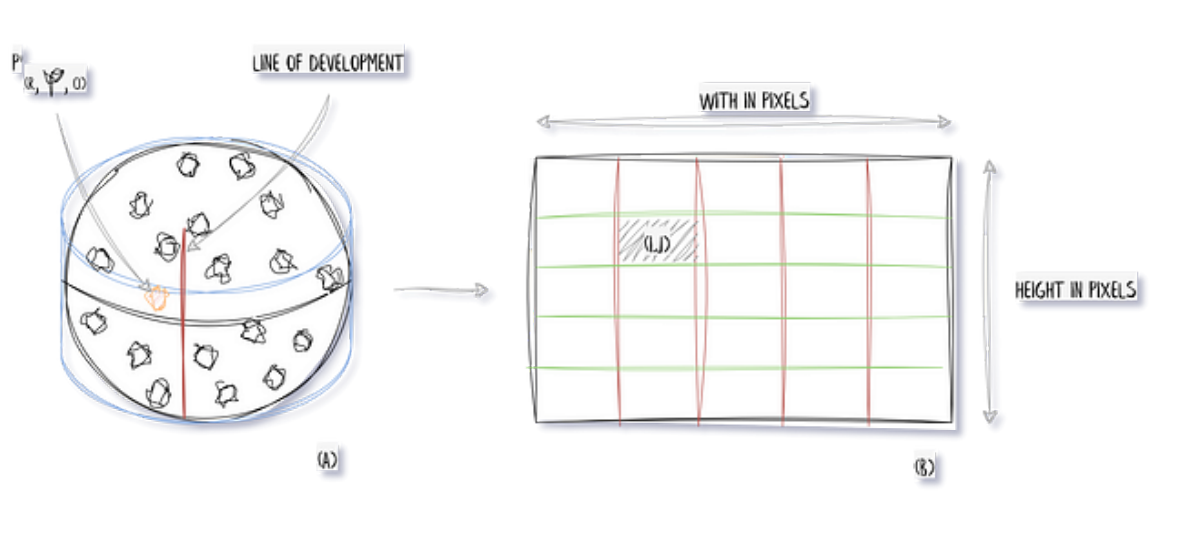
The 3D Point Cloud Spherical Projection Workflow. We take a 3D Point Cloud, we create a 3D Projection Sphere, we define the mapping plane, and we produce an equirectangular projection. © F. Poux

To achieve the 3D Projection onto a sphere, we want to obtain points as illustrated below.



How to project points of a 3D point clouds onto a sphere. © F. Poux

Then, we will unroll following our geometry (cylinder) to obtain an equirectangular image, as shown below.



How to go from a sphere to an equirectangular image. We have a projection mechanism that permits us to "unroll" the pixels onto a cylinder. © F. Poux

Let me now detail the function that allows just this:

```python
def generate_spherical_image(center_coordinates, point_cloud, colors, resolu
    # Translate the point cloud by the negation of the center coordinates
    translated_points = point_cloud - center_coordinates

    # Convert 3D point cloud to spherical coordinates
    theta = np.arctan2(translated_points[:, 1], translated_points[:, 0])
    phi = np.arccos(translated_points[:, 2] / np.linalg.norm(translated_poin

    # Map spherical coordinates to pixel coordinates
    x = (theta + np.pi) / (2 * np.pi) * (2 * resolution_y)
    y = phi / np.pi * resolution_y

     # Create the spherical image with RGB channels
    resolution_x = 2 * resolution_y
    image = np.zeros((resolution_y, resolution_x, 3), dtype=np.uint8)

    # Create the mapping between point cloud and image coordinates
    mapping = np.full((resolution_y, resolution_x), -1, dtype=int)

    # Assign points to the image pixels
    for i in range(len(translated_points)):
        ix = np.clip(int(x[i]), 0, resolution_x - 1)
        iy = np.clip(int(y[i]), 0, resolution_y - 1)
        if mapping[iy, ix] == -1 or np.linalg.norm(translated_points[i]) < n
            mapping[iy, ix] = i
            image[iy, ix] = colors[i]
    return image
```

🌱 **Growing:** *It is essential to digest this function. It looks like it is pretty straightforward, but there are nice tricks at several stages. For example, what do you think about the 3D point cloud to spherical coordinates step? What does the mapping do? What is the point of using the mapping as a conditional statement while assigning points to pixels?*

Now, to use this handy function, let us load and prepare the ITC indoor point cloud first:

```python
#Loading the las file from the disk
las = laspy.read("../DATA/ITC_BUILDING.las")

#Transforming to a numpy array
coords = np.vstack((las.x, las.y, las.z))
point_cloud = coords.transpose()

#Gathering the colors
r=(las.red/65535*255).astype(int)
g=(las.green/65535*255).astype(int)
b=(las.blue/65535*255).astype(int)
colors = np.vstack((r,g,b)).transpose()
```

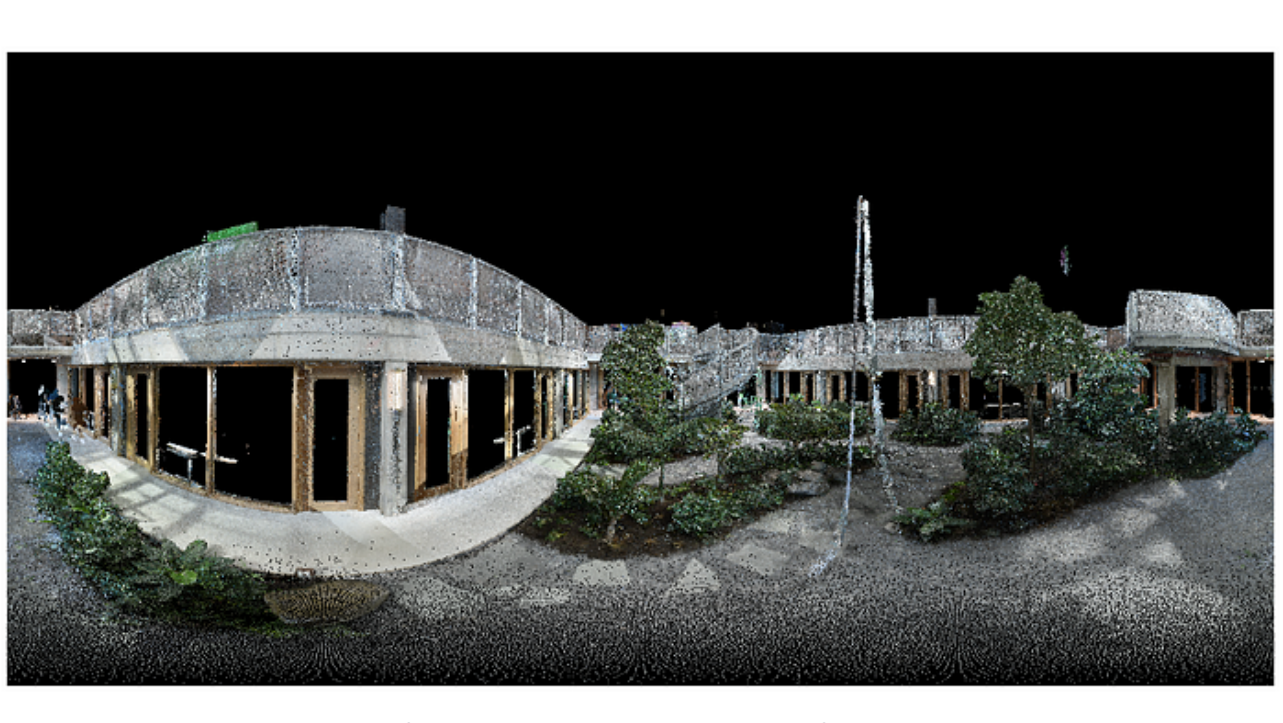
Once prepared, we can define the necessary parameters for projection. These are the center of projection (basically the position from which we want a virtual scan station) and the resolution of the final image (expressed in pixels, as the smallest side of the image).

```python
resolution = 500

#Defining the position in the point cloud to generate a panorama
center_coordinates = [189, 60, 2]
```

Finally, we can call the new function, plot and export the results as an image

```python
#Function Execution
spherical_image, mapping = generate_spherical_image(center_coordinates, poin

#Plotting with matplotlib
fig = plt.figure(figsize=(np.shape(spherical_image)[1]/72, np.shape(spherica
fig.add_axes([0,0,1,1])
plt.imshow(spherical_image)
plt.axis('off')

#Saving to the disk
plt.savefig("../DATA/ITC_BUILDING_spherical_projection.jpg")
```

All this process results in the following image:



The 3D point cloud transformed as an equirectangular image from the projection. © F. Poux

How do you like that? You can play around with the various parameters, such as the resolution or the center of projection, to ensure that you get a nice balance between "no data" pixels and relevant panorama.

🦊 **Florent:** *You just unlocked a powerful new skill with 3D Point Cloud to Equirectangular image creation. Indeed, it allows you to generate virtual scans basically wherever you believe it makes sense and then unlock the possibility to use image processing and deep learning techniques for images. You can also extend the provided function to other mapping projections to add arrows to your quiver.*

🤠**Ville:** *I can almost see the lecture halls and my office, Dutch working vibes!*

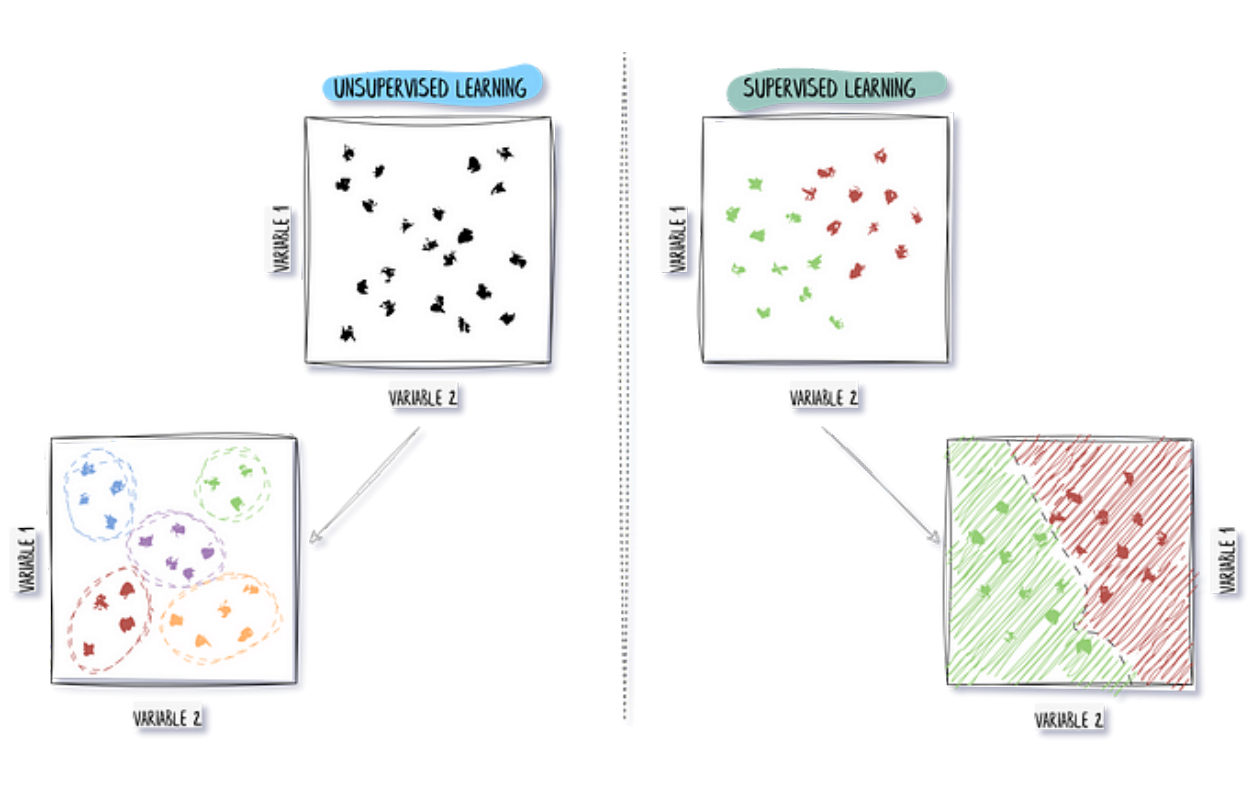
3.3 3D Point-to-Pixel Mapping

We transform raw point data into structured raster representations, making sense of the seemingly scattered information. Point Cloud Mapping is the compass that guides us for 3D point cloud processing through 2D projection. The good news: we already took care of this mapping.

Indeed, if you take a close look at the function `generate_spherical_image`, you can see that we return the `mapping` variable and capture it in another variable for downward processes. This ensures that we can have a coherent 3D Point-to-Pixel Mapping.

4. Unsupervised Segmentation with SAM

Unsupervised segmentation enters the scene in the form of the Segment Anything Model. We are, in the case of non-labeled outputs, through SAM's segmentation architecture, which falls within clustering applications. This is opposed to most supervised learning approaches that will provide labeled outputs, as illustrated below.



The distinction between unsupervised learning and supervised learning. In unsupervised learning, we aim at defining groups of data "points" that share some similarity, whereas in supervised learning, we aim at approaching the supervision needs (usually by feeding labeled data). © F. Poux

Therefore, the transfer of pixel predictions, coupled with seamless point cloud export, showcases the potential for revolutionizing applications like object detection and scene understanding.

4.1. SAM Segmentation

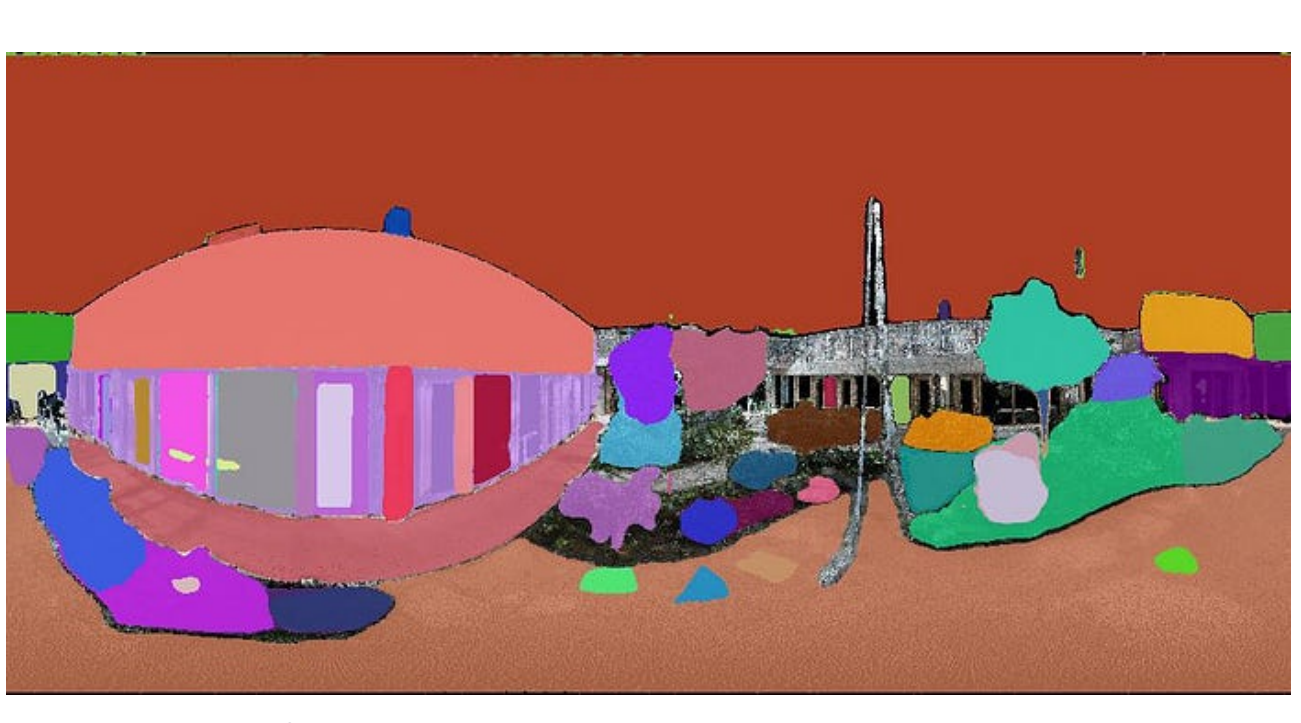
To execute the program, we can re-execute the code snippets that we used to test our SAM functionalities on 2D images, which are:

```python
sam = sam_model_registry["vit_h"](checkpoint = MODEL)
sam.to(device = USED_D)

mask_generator = SamAutomaticMaskGenerator(sam)

temp_img = cv2.imread("../DATA/ITC_BUILDING_spherical_projection.jpg")
image_rgb = cv2.cvtColor(temp_img, cv2.COLOR_BGR2RGB)

t0 = time.time()
result = mask_generator.generate(image_rgb)
t1 = time.time()
```

and later, we can plot the results on the image itself

```python
fig = plt.figure(figsize=(np.shape(image_rgb)[1]/72, np.shape(image_rgb)[0]/
fig.add_axes([0,0,1,1])

plt.imshow(image_rgb)
color_mask = sam_masks(result)
plt.axis('off')
plt.savefig("../DATA/ITC_BUILDING_spherical_projection_segmented.jpg")
```

Which results in:



Results of the Segment Anything Model on the 3D point cloud projection. © F. Poux

This already looks like we are delineating significant parts of the image. Let us move forward with this.

4.2. Point Prediction Transfer

Let us color the point cloud with this image. We thus define a coloring function:

```python
def color_point_cloud(image_path, point_cloud, mapping):
    image = cv2.imread(image_path)
    h, w = image.shape[:2]
    modified_point_cloud = np.zeros((point_cloud.shape[0], point_cloud.shape
    modified_point_cloud[:, :3] = point_cloud
    for iy in range(h):
        for ix in range(w):
            point_index = mapping[iy, ix]
            if point_index != -1:
                color = image[iy, ix]
                modified_point_cloud[point_index, 3:] = color
    return modified_point_cloud
```

This means that to color our point cloud, we can use the following code line that calls our new function:

```python
modified_point_cloud = color_point_cloud(image_path, point_cloud, mapping)
```

This line returns a numpy array that holds the point cloud.

It is now time for 3D Point Cloud Export!

4.3. Point Cloud Export

To export the point cloud, you can use numpy or laspy to extract a .las file directly. We will proceed with the second solution:

```python
def export_point_cloud(cloud_path, modified_point_cloud):
    # 1. Create a new header
    header = laspy.LasHeader(point_format=3, version="1.2")
    header.add_extra_dim(laspy.ExtraBytesParams(name="random", type=np.int32

    # 2. Create a Las
    las_o = laspy.LasData(header)
    las_o.x = modified_point_cloud[:,0]
    las_o.y = modified_point_cloud[:,1]
    las_o.z = modified_point_cloud[:,2]
    las_o.red = modified_point_cloud[:,3]
    las_o.green = modified_point_cloud[:,4]
    las_o.blue = modified_point_cloud[:,5]
    las_o.write(cloud_path)

    print("Export succesful at: ", cloud_path)
    return
```

And with this, we can export our modified_point_cloud variable:

```python
export_point_cloud("../DATA/pcd_results.las", modified_point_cloud)
```

After this stage, we successfully taken our various 2D images resulting from the 3D point cloud projection process. We applied SAM algorithm to it, colorized it based on its prediction, and exported a colored point cloud. We can thus move to getting some insights about what we are getting.

🦊 **Florent:** *To analyze the result quickly outside Python, I recommend using the CloudCompare Open-Source Software. If you want a clear guide on how to use it efficiently, you can read and follow through the article below.*

3D Deep Learning Python Tutorial: PointNet Data Preparation

The Ultimate Python Guide to structure large LiDAR point cloud for training a 3D Deep Learning Semantic Segmentation...

towardsdatascience.com

5. Qualitative Analysis and discussions

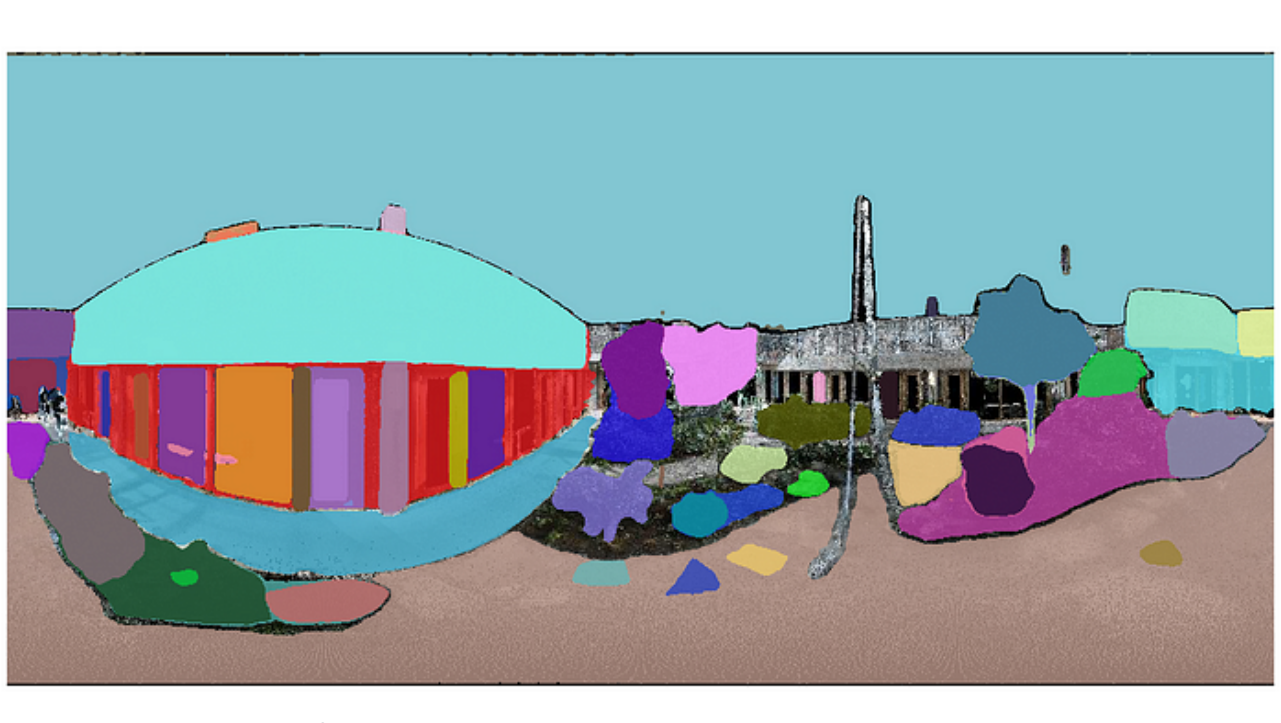
With our journey nearing its zenith, it's time to focus on qualitative analysis. Exceptionally, we will not conduct a quantitative analysis, as we would need proper labels for that at this stage.

🤠 **Ville:** No labels? What you just did was zero-shot learning (Bang!) or few-shot learning (Bang! Bang!). We cannot be sure which because we don't know exactly how SAM was trained by Meta. Therefore it is a bit of a black box for us, but that's ok.

We meticulously examine the raster and point cloud results, drawing insights that shed light on SAM's performance. Also, let us remain grounded by acknowledging the model's limitations while setting our sights on the future.

5.1 Raster results

The output of SAM's efforts with our implementation is eloquently depicted through the below raster results. These visuals serve as a canvas on which SAM's segmentation can be quickly assessed, enabling us to comprehend the model's understanding of the scene in a 2D representation.

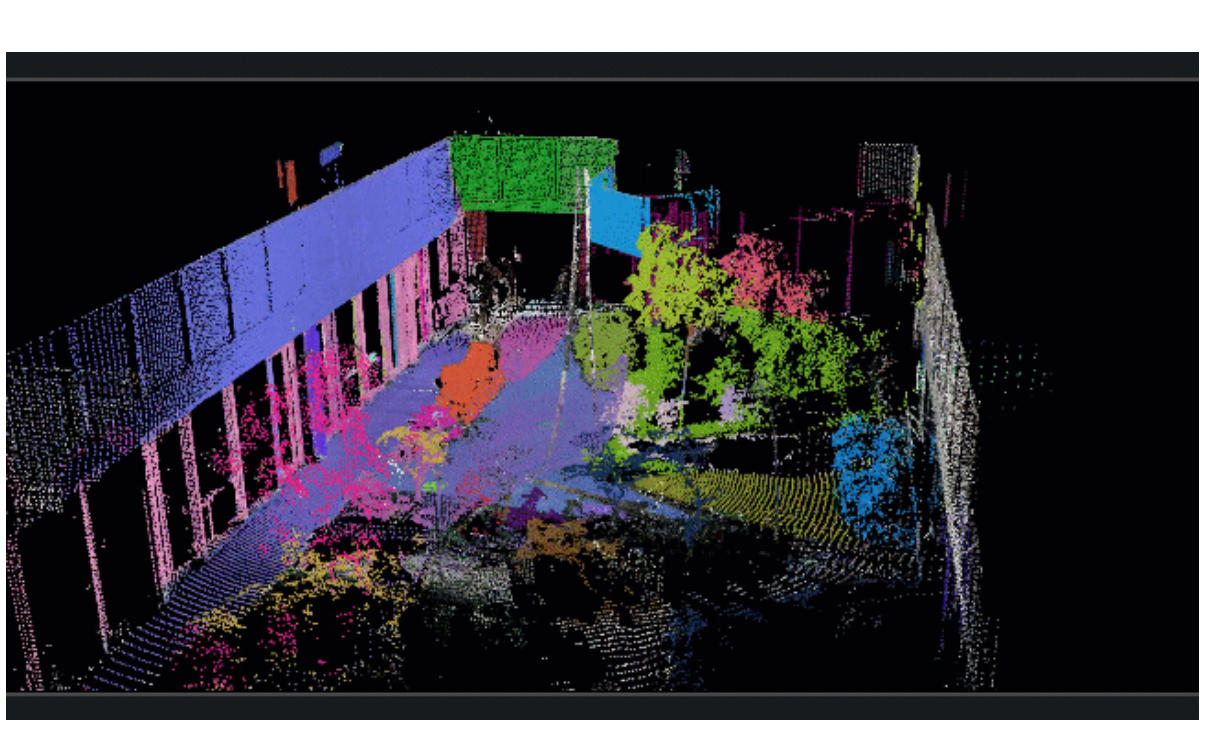


Another result of the Segment Anything Model on the 3D point cloud projection. © F. Poux

As you can see, even with the uneven point distribution and "black zones", SAM is able to pick up what the main parts of the point cloud are about. Specifically, it likely highlights a green on the left, the place where the dangerous materials are, and the doors and windows for our extraction team to have the most direct route!

5.2. Point Cloud results

Yet, it's in the point cloud results that the true depth of SAM's abilities emerges. As we navigate through the cloud of points, SAM's segmented predictions bring clarity to the classical "mess of points", showcasing its potential in real-world applications.



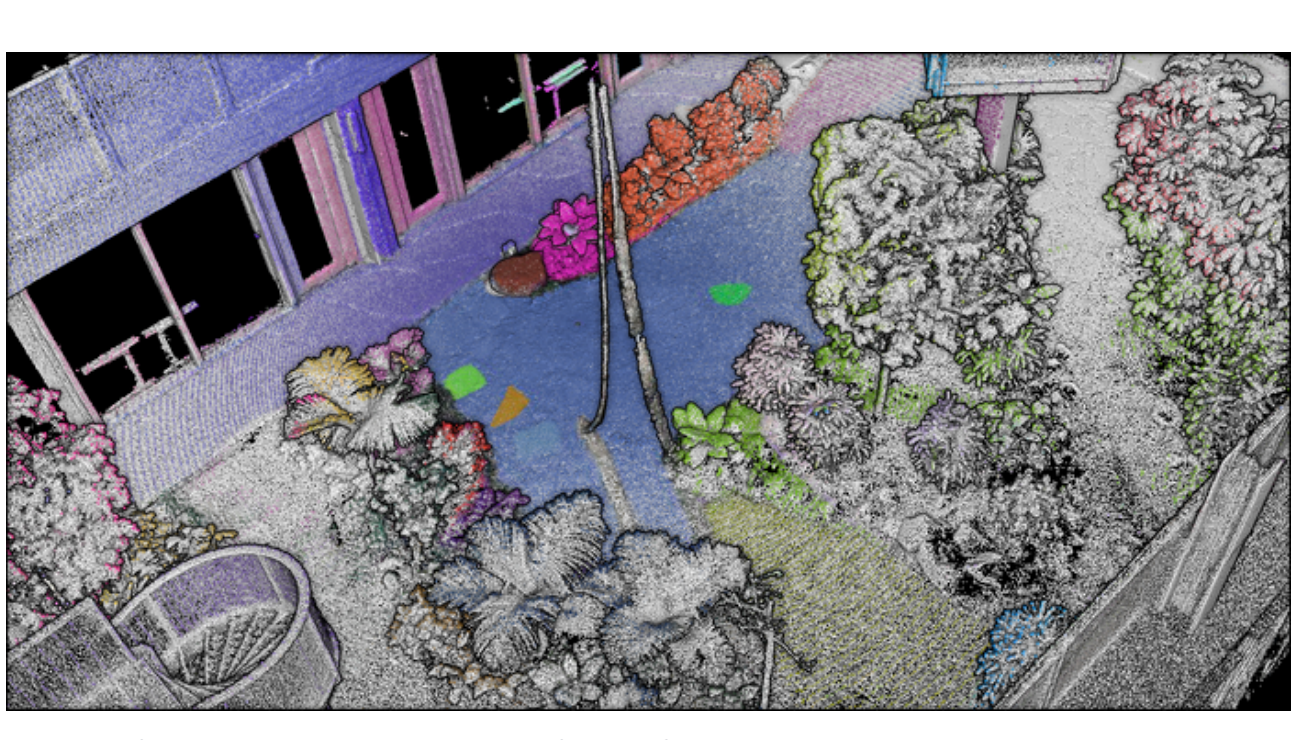
The 3D Point Cloud unsupervised segmentation results by using Segment Anything 3D. We see the great distinction of the major elements that compose the scene. © F. Poux

As we can see, we can have a direct link with the underlying points, and that is massively awesome! Think only about what this can unlock for your applications. A 100% automated segmentation process that has under five main breakpoints? Not bad!

5.3. Shortcomings

But, our expedition would only be complete with acknowledging the rough patches along the way. SAM, while impressive, is not exempt from limitations. By recognizing these shortcomings, we pave the way for refinement and growth

The first thing is that all the "unseen" points remain unlabelled (white points below). This could prove to be a limitation for complete processing, and if you use the basic or the large model you will see more unlabelled points than when using the huge model.



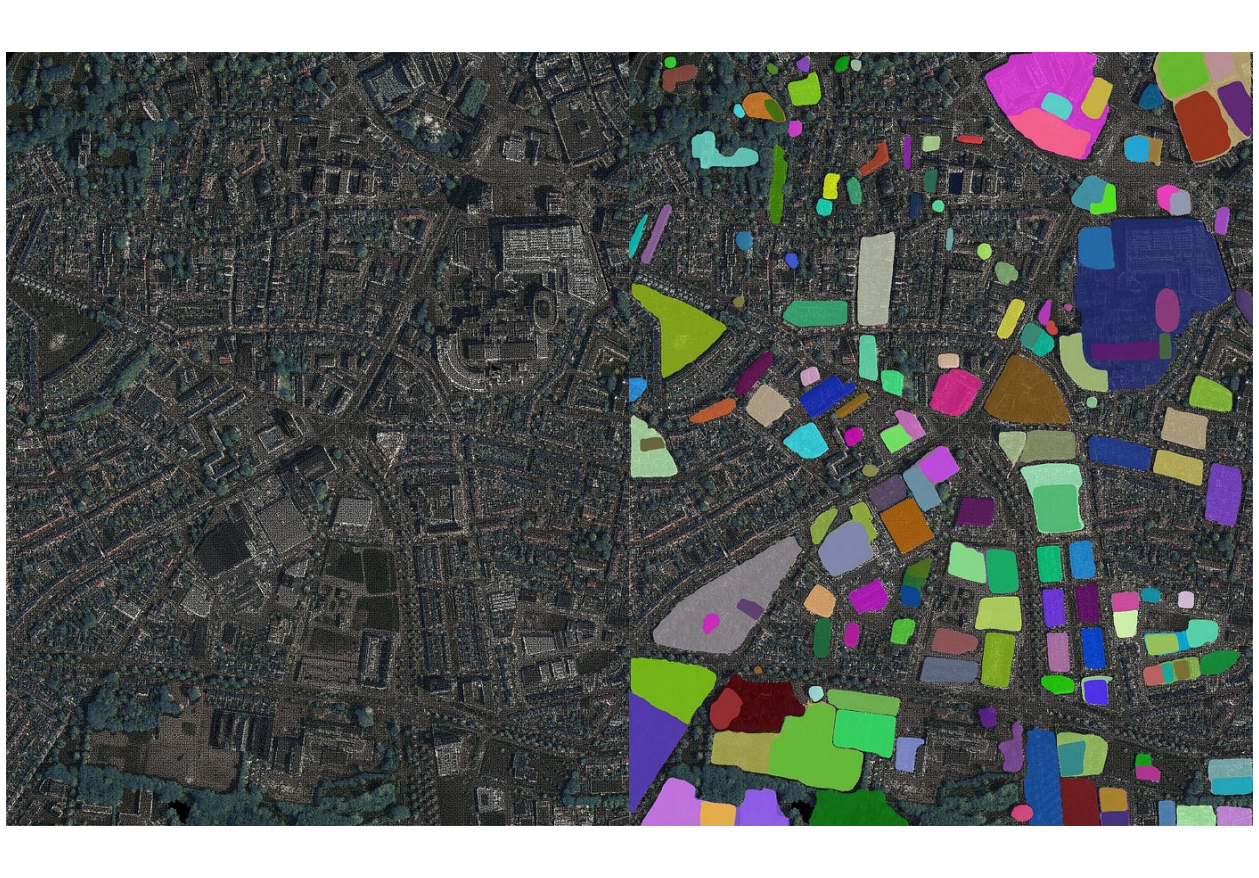
The ratio of unlabelled points vs. labeled points from the first pass with a central perspective 360° simulated scan position. © F. Poux

Also, at this stage, we used the automatic prompting engine that triggered around 50 points of interest, the seeds of the segmentation task. While this is great for getting a direct result, having the possibility to tune that would be awesome.

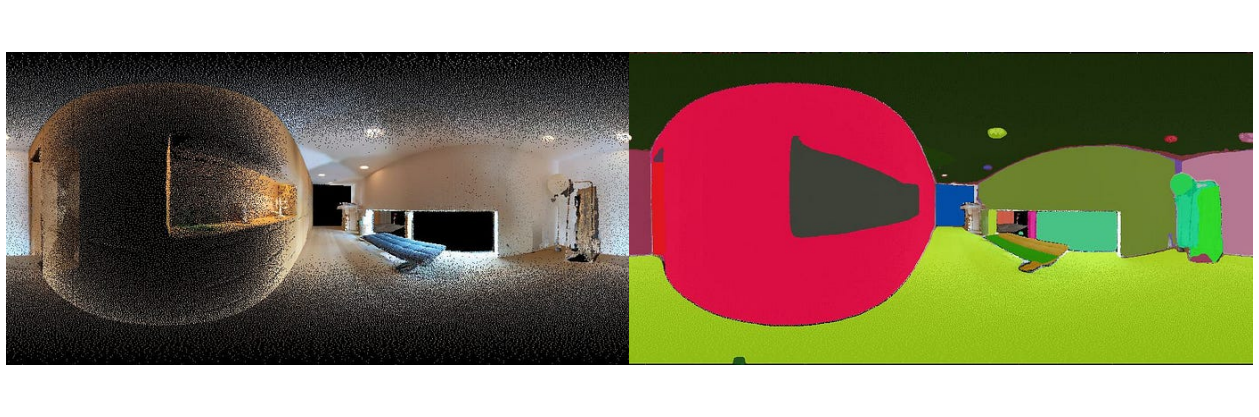
Finally, the mapping is somewhat simple at this stage; it would largely benefit from occlusion culling and point selection for a specific pixel of interest.

5.4. Perspectives

The Segment Anything Model marks just a single step in the larger landscape of 3D point cloud segmentation. However, as it stands and is given to you, our implementation should work pretty well for any application where you can have some kind of distinctive initial features for SAM. As you can see below, it also works for top-down aerial point clouds.



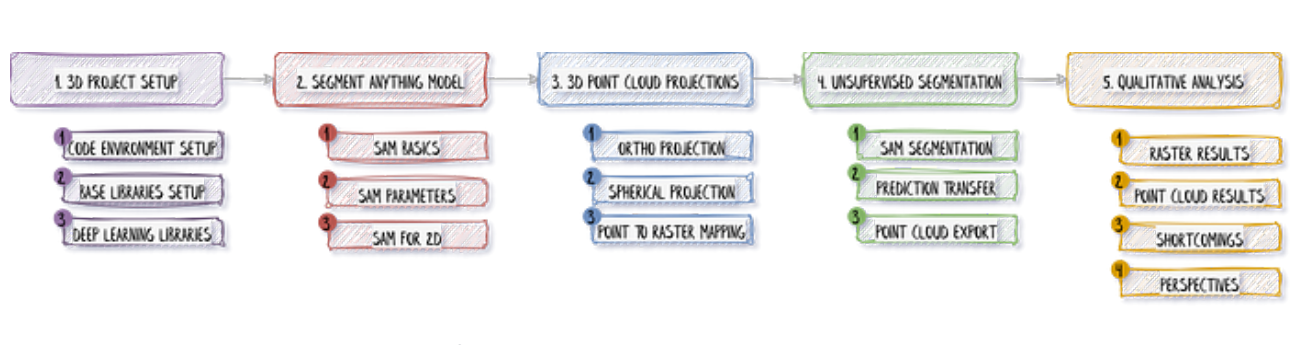
Extending to indoor scenarios, you can see that you also will get some pretty decent and interesting results. This is even useful for changing the light bulb of the light fixtures in the hall, automatically by a robot, of course (How many robots does it take to change a light bulb?)!



Therefore, aside from generalization, one first perspective is to unlock a way to generate panoramas and fuse the prediction of the different points of view. Of course, another one would be to expand to custom prompts and, finally, address the challenge of improving point-to-pixel accuracy in 2D-3D mapping.

Conclusion

If you are part of the 13.37% of 3D creators that went ahead and actually made the code work, then massive kudos to you!



The workflow that we covered in this article. © F. Poux

This is a tremendous achievement, and you now have a very powerful asset for attacking semantic extraction tasks for 3D Scene Understanding. With the Segment Anything Model, you can now encapsulate innovation in many products, transforming how we perceive and interpret 3D point clouds.

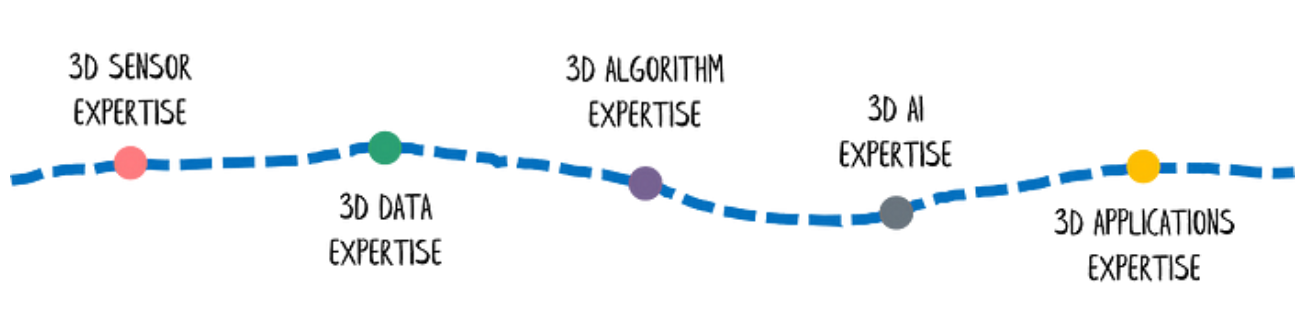
Our exploration should have painted a comprehensive, usable picture of this groundbreaking model from its inception to its implications. You may now explore the variants and extend their pertinency based on the limitations that were spotted in the previous parts.

🦊 **Florent:** *I am looking forward to your future projects that make use of it!*

🤠 **Ville:** *Code on!*

References

1. Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y. and Dollár, P., 2023. Segment anything. *arXiv preprint arXiv:2304.02643*.
2. **Poux, Florent**, Mattes, C., Selman, Z. and Kobbelt, L., 2022. Automatic region-growing system for the segmentation of large point clouds. *Automation in Construction*, *138*, p.104250. Elsevier Link
3. **Lehtola, Ville**, Kaartinen, H., Nüchter, A., Kaijaluoto, R., Kukko, A., Litkey, P., Honkavaara, E., Rosnell, T., Vaaja, M.T., Virtanen, J.P. and Kurkela, M., 2017. Comparison of the selected state-of-the-art 3D indoor scanning and point cloud generation methods. *Remote sensing*, *9*(8), p.796. MDPI Link



🔷Other Resources

- 🍇 Get Access to the Data here: 3D Datasets
- 👸 3D Online Data Science Courses: 3D Academy
- 📖 Subscribe for early access to 3D Tutorials: 3D AI Automation
- 👨‍🎓Get a Master's Degree: ITC Utwente

🎓Author's Recommendation

To build full Indoor Semantic Extraction Scenarios, you can combine this approach with the one explained in the "*3D Point Cloud Shape Detection for Indoor Modelling*" article:

3D Point Cloud Shape Detection for Indoor Modelling

A 10-step Python Guide to Automate 3D Shape Detection, Segmentation, Clustering, and Voxelization for Space Occupancy...

towardsdatascience.com

#3d #python #segmentation #point-cloud #deep-dives